# AUTOBOTs: Autonomous Security Engineer

**Bilal Aslam - 2022361**

**Muhammad Bilal - 2022359**

**Syed Ahmed Haseeb - 2022557**

**Muhammad Hassaan Shah - 2022385**

**Advisor: Hafiz Syed Ahmed Qasim**
**Co-Advisor: Muhammad Ahmad Nawaz**

**Faculty of Computer Science and Engineering (FCSE)**
**GIK Institute of Engineering Sciences and Technology**

## Certificate of Approval

It is certified that the work presented in this report was performed by **Syed Ahmed Haseeb, Muhammad Hassaan Shah, Muhammad Bilal, and Bilal Aslam** under the supervision of **Hafiz Syed Ahmed Qasim** and **Muhammad Ahmad Nawaz**. The work is adequate and lies within the scope of the BS degree in Cyber Security at Ghulam Ishaq Khan Institute of Engineering Sciences and Technology.

---
**Hafiz Syed Ahmed Qasim**
(Advisor)

---
**Muhammad Ahmad Nawaz**
(Co-Advisor)

---
**Prof. Dr. Qadeer Ul Hassan**
(Dean)

## SDGs Inclusion

**SDG 3: Good Health / Wellbeing**

AUTOBOTs helps in health and well-being by securing sensitive medical data and important systems like healthcare platforms. Cyberattacks on healthcare systems may affect services and disrupt patient safety, thus cybersecurity is vital. By identifying weaknesses early, AUTOBOTs will prevent system failures and data breaches. It also lowers the pressure on cybersecurity teams through automating intricate tasks and cutting the workload. This enables professionals to concentrate on making strategic decisions as opposed to monotonous tasks. Moreover, it can contribute to trust in digital health systems by ensuring the privacy of data and the reliability of the system. Altogether, AUTOBOTs helps to create safer and more trustworthy digital spaces that help to enhance the welfare of society.

**SDG 8: Decent Work and Economic Growth**

AUTOBOTs also helps to grow the economy since over-reliance on limited and costly cybersecurity experts is limited. As expenses continue to rise with H-1B recruitment and the need to employ on-site security personnel, organizations are experiencing a great financial burden. AUTOBOTs offers a cheaper substitute with automation, companies can uphold high security without having to increase their staff. It also enhances productivity by automating repetitive and time-consuming tasks like scanning and reporting. The system enhances quicker and more secure business practices by enhancing efficiency and reducing the time lag in remediation. It also gives skilled professionals the chance to work on more advanced security strategy as opposed to manual testing. In general, AUTOBOTs facilitates sustainable and effective workforce.

**SDG 9: Industry, Innovation and Infrastructure**

AUTOBOTs directly contributes to the field of innovation, being an innovative AI-driven multi-agent cybersecurity framework. It maximizes digital infrastructure resilience by actively finding and addressing vulnerabilities in the web and cloud environments. The system takes advantage of the latest technologies like LLMs, autonomous agents, and scalable architectures to transform conventional penetration testing. It enhances secure software development practices, by being seamlessly integrated into development pipelines and IDEs. AUTOBOTs also assists organizations to develop strong, secure digital systems capable of staying up to date with the changes in cyber threats. It can be scaled to be deployed in enterprises of different sizes. It helps to promote safe and innovative technology ecologies.

**SDG 16: Peace, Justice and Strong Institutions**

Cybersecurity is essential in providing trust, privacy and integrity. This is supported by AUTOBOTs, which can help organizations to protect sensitive data and avert cyberattacks that can compromise systems or confidence of people. It promotes transparency and accountability by making sure that standards, including PCI DSS, HIPAA, and ISO 27001 are adhered to. The system minimizes the chances of data breach, financial fraud, and unauthorized access which is essential in ensuring stable institutions. It also promotes responsible and ethical penetration testing practices. AUTOBOTs ensures to make digital environments safer by making it possible to detect threats proactively. This eventually helps in the development of stronger and more secure institutions.

| | |
|---|---|
| --- | --- |
| **Hafiz Syed Ahmed Qasim** | **Muhammad Ahmad Nawaz** |
| (Advisor) | (Co-Advisor) |

# ABSTRACT

Today, cybersecurity is one of the most critical concerns for any organization and its customers. Security has always been an important aspect of any digital system, but the increasing interconnectedness and the ever more sophisticated cyber-attacks make security concerns a top priority. Traditional security mechanisms such as firewalls, intrusion detection systems, and antivirus software are still important, but they are no longer sufficient to protect complex software systems from modern threats.

This project introduces AUTOBOTs: Autonomous Security Engineer, an AI-driven cybersecurity framework that automates and improves penetration testing and vulnerability management. AUTOBOTs utilizes a multi-agent architecture with embedded Large Language Models (LLMs) and vulnerability intelligence to conduct the complete security operations from reconnaissance, vulnerability discovery, exploit development, proof-of-concept validation, to remediation. With contextual information reservation over multi-step testing campaigns and intelligent decision mechanism, AUTOBOTs minimizes false positives giving precise and actionable outcomes.

AUTOBOTs facilitates continuous security assessment in dynamic settings and allows for scalable, parallel testing across numerous targets. Additionally, it has compliance-aware reporting that complies with ISO/IEC 27001:2022, HIPAA, and PCI DSS 4.0 standards. By bringing a proactive, intelligent, and autonomous approach to vulnerability assessment, this project seeks to rethink conventional cybersecurity processes and help organizations identify, validate, and mitigate threats more effectively and consistently.

# ACKNOWLEDGEMENTS

This achievement would not have been possible without the assistance, encouragement, and mentorship provided by several people and our educational institution. Our sincere gratitude goes out to everyone concerned with the project.

We would first like to thank our advisor, Hafiz Syed Ahmed Qasim, who played an important role in guiding us through the process of this project. He assisted us in coming up with new concepts regarding our objectives and his counseling was a major factor in different periods of this research. Similarly, we should also give equal recognition to our co-advisor Muhammad Ahmad Nawaz, who was always guiding us to ensure that we would deliver this task in time.

We owe our university, Ghulam Ishaq Khan Institute of Engineering Sciences and Technology, the academic atmosphere and facilities that we needed to conduct this research work. The high-level of academic excellence maintained by the institution has always encouraged us towards research and practical based studies.

Lastly, we acknowledge our friends and colleagues who helped us by suggesting their views about this project and by sharing their valuable experiences with us. Their involvement in the project has proved beyond expectation.

## TABLE OF CONTENTS

**CHAPTER I ..... 1**

1. INTRODUCTION ..... 1
   1.1. Objectives ..... 3
   1.2. Motivation ..... 4
   1.3. Project's Scope ..... 5

**CHAPTER II ..... 8**

2. Literature Survey ..... 8

**CHAPTER III ..... 12**

3. System Design ..... 12
   3.1. Model Description ..... 13
   3.2. System Requirements ..... 15
   3.3. Software Functionality ..... 18
   3.4. User Characteristics ..... 20
   3.5. Constraints ..... 22
   3.6. Assumptions and Dependencies ..... 24
   3.7. Functional Requirements ..... 25
   3.8. Nonfunctional Requirements ..... 30
   3.9. Project Design/Architecture ..... 33

**CHAPTER IV ..... 45**

4. Proposed Solution ..... 45
   4.1. Methodology ..... 45
   4.2. Implementation ..... 48
   4.3. Competitive Advantages Over Existing Solutions ..... 51

**CHAPTER V ..... 52**

5. Results and Discussion ..... 52
   5.1. Execution Time Performance ..... 55
   5.2. Vulnerability Detection Quality ..... 57
   5.3. CTF and Bug Bounty Performance ..... 58
   5.4. System Responsiveness and Non-Functional Performance ..... 59
   5.5. Comparison with Existing Systems ..... 60

**CHAPTER VI ..... 64**

6. Conclusion and Future Work ..... 64
   6.1. Conclusion ..... 64
   6.2. Future Work ..... 67

**GLOSSARY ..... 69**

**REFERENCES** ..... **72**
**APPENDIX A** ..... **74**
FUNCTIONAL REQUIREMENTS WITH TRACEABILITY INFORMATION ..... 74
**APPENDIX B** ..... **79**
TOOL INTEGRATION REFERENCE ..... 79

## LIST OF FIGURES

FIGURE 1: ARCHITECTURAL DESIGN ..... 34
FIGURE 2: USE CASE VIEW ..... 35
FIGURE 3: LOGICAL VIEW ..... 35
FIGURE 4: COMPONENTS DIAGRAM ..... 36
FIGURE 5: DEVELOPMENT VIEW ..... 37
FIGURE 6: CONTEXT DIAGRAM ..... 38
FIGURE 7: PROCESS VIEW ..... 39
FIGURE 8: SEQUENCE DIAGRAM ..... 40
FIGURE 9: ACTIVITY DIAGRAM ..... 41
FIGURE 10: PHYSICAL VIEW ..... 42
FIGURE 11: AUTOBOTS CLI ..... 43
FIGURE 12: PROMPT ..... 44
FIGURE 13: TOOL CONFIRMATION IN CLI ..... 44
FIGURE 14: PERFORMANCE METRICS ..... 52
FIGURE 15: A SUCCESSFUL RUN OF AUTOBOTS ..... 53
FIGURE 16: BEGINNING OF REPORT ..... 54
FIGURE 17: ATTACK CHAIN ..... 54
FIGURE 18: REMEDIATIONS ..... 54
FIGURE 19: EXECUTION TIME COMPARISON ..... 56
FIGURE 20: FALSE POSITIVES COMPARISON ..... 58
FIGURE 21: SPEEDUP FACTOR OVER MANUAL PEN-TESTING ..... 61
FIGURE 22: KEY METRICS ..... 62
FIGURE 23: HOLISTIC PERFORMANCE RADAR ..... 63

## LIST OF TABLES

**TABLE 1: KEY FEATURES OF AUTOBOTS** ..... 6
**TABLE 2: LITERATURE SURVEY RECAP** ..... 11
**TABLE 3: USER CHARACTERISTICS** ..... 21
**TABLE 4: EXECUTION TIME COMPARISON** ..... 56
**TABLE 5: FUNCTIONAL REQUIREMENTS 1** ..... 75
**TABLE 6: FUNCTIONAL REQUIREMENTS 2** ..... 75
**TABLE 7: FUNCTIONAL REQUIREMENTS 3** ..... 76
**TABLE 8: FUNCTIONAL REQUIREMENTS 4** ..... 77
**TABLE 9: FUNCTIONAL REQUIREMENTS 5** ..... 77
**TABLE 10: FUNCTIONAL REQUIREMENTS 6** ..... 78

# CHAPTER I

# 1. INTRODUCTION

In today's highly volatile and fast-paced digital world, cybersecurity threats have increased in terms of scope, intensity, and frequency. Conventional security practices, including regular security checks, penetration testing, and patching, no longer offer adequate protection against cyber threats and risks associated with modern applications running in a continuous delivery environment. Organizations face challenges related to development speed, increased complexity, and new vulnerabilities that arise from rapid changes.

A cybersecurity breach causes losses, negative publicity, and other issues that may be exacerbated by conventional manual security checks and existing tools' slow response times. The latter's performance is also negatively impacted by the high number of false positives and difficulty dealing with new types of cyber threats. Therefore, there is a growing need for autonomous penetration testing that can identify, validate, and fix any vulnerabilities in a timely manner. AUTOBOTs is intended to fulfill this purpose by creating a multi-agent security infrastructure that will perform intelligent, real-time, and scalable offensive security operations without human intervention.

AUTOBOTs: Autonomous Security Engineer attempts to solve these problems using a framework for security testing that combines artificial intelligence, autonomous behavior, and adaptability. The approach uses a multi-agent system with capabilities that include using LLMs, vulnerability intelligence, and secure software engineering techniques to execute

penetration tests, proof-of-concept testing, and risk mitigation actions autonomously.

Unlike conventional automation tools, AUTOBOTs are based on intelligent decision making and human-like reasoning to make certain that test campaigns preserve the context and are used across multiple stages and reduce false positives and are scalable enough to support many end points. Using this method, the system can effectively bridge the gap between offensive security and continuous protection, enabling enterprises to meet the standards such as PCI DSS 4.0, HIPAA, and ISO/IEC 27001:2022 every time and at the same time accelerate the delivery of software.

The goal of this paper is to provide a comprehensive overview of the AUTOBOTs: Autonomous Security Engineer project in detail, such as project purpose, design aspects, methodology of implementation used, and project outcomes. The structure of this paper will be designed systematically so that each key issue related to the project can be discussed thoroughly starting from an introduction about the subject matter, then a review of past literature on it, an elaboration on the specific designs and features of the system, the actual solution to the issue and how it is implemented, and finally the results and future directions.

## 1.1. Objectives:

The main aim of AUTOBOTs is to develop and implement an autonomous cybersecurity system that will be able to conduct full penetration testing with minimal human intervention. The system aims to automate several crucial steps of the security evaluation process, such as reconnaissance, vulnerability scanning, exploitation, proof-of-concept, and remedy recommendations. With the help of multiple agents and LLMs, AUTOBOTs attempt to introduce adaptive reasoning and context-sensitive decision-making in procedures that are typically performed manually.

One of the goals of this project is to increase the precision of automated security testing by minimizing false positives using reproducible validation of exploits. This guarantees that any discovered vulnerabilities can be reproduced and therefore exploited. Moreover, the use of AUTOBOTs allows one to maintain context throughout multiple stages of attacks, thereby providing the possibility for chain analysis and prolonged testing processes.

The purpose of this system is to provide scalability by conducting tests on multiple targets and environments simultaneously, thus fulfilling the needs of enterprise-level implementation. Another feature that makes AUTOBOT highly efficient is its focus on reducing the amount of human intervention required for the process, fastening the identification of vulnerabilities and fixing them. Moreover, the system is aimed at supporting the adherence to the commonly accepted security models like PCI DSS, HIPAA, and ISO/IEC 27001.

AUTOBOTs is going to abandon the old-fashioned approach to cybersecurity that is not proactive and is too human-dependent in favor of an intelligent, more effective, and independent one.

## 1.2. Motivation:

The motivation to come up with AUTOBOTs is the growing level of complexity and size of the software systems in use today, since they have introduced additional layers to the threat of cybersecurity. Cloud computing, microservices, and CI/CD (Continuous Integration & Continuous Deployment) have opened new loopholes in the cybersecurity infrastructure of software systems, making them vulnerable to cyber-attacks. Conventional approaches towards security like manual penetration testing and vulnerability assessment are no longer enough to tackle today's constantly changing cyber threats.

Manual penetration testing is very expensive in terms of finances, time, and highly skilled staff, which are scarce. Even with the effectiveness of the current automation tools, the amount of expertise to analyze the data received is usually lacking, resulting in the high percentage of false positives, and inability to detect more complex vulnerabilities, such as business logic errors and multi-step attacks.

The AUTOBOTs initiative is motivated by the goal of tackling these issues by incorporating artificial intelligence and automation in cybersecurity operations. This is done by enabling continuous, intelligent and adaptive cybersecurity testing, which is not only human free, but also highly precise and can be scaled to suit the current digital contexts.

The AUTOBOTs initiative is motivated by the goal of tackling these issues by incorporating artificial intelligence and automation in cybersecurity operations. This is done by enabling a continuous, intelligent, and

adaptive cybersecurity testing, which is not only human free, but also highly precise and can be scaled to suit current digital contexts.

### 1.3. Project's Scope:

The scope of AUTOBOTs includes creating an autonomous and AI-powered platform for penetration testing and vulnerability management that is set up to work with web applications, APIs, and cloud-based systems. This platform is expected to be able to accomplish a broad spectrum of tasks related to cybersecurity, including OSINT and reconnaissance, vulnerability scanning, exploitation, authentication testing, and cloud security testing.

AUTOBOTs utilizes a multi-agent architecture, in which various kinds of agents work together to accomplish particular stages of penetration testing. The coordination of agents are orchestrated by an orchestration layer that renders the testing process to be intelligent, adaptable, and context-aware so that context information is maintained across different multi-step and complex test processes. Evidence of concept tools are applied to verify that vulnerabilities that are identified are genuine.

The objective of AUTOBOTs is to offer scalability and parallel testing of numerous targets, which enables deploying the product in both small and large businesses. The tool works in combination with third-party security products, databases, and application programming interfaces in order to enhance its analysis and keep up-to-date information about threats. Additionally, the tool features advanced reporting, with detailed structured reports including information about potential vulnerabilities, impacts, and their respective remedies.

This includes compliance with such industry regulations as PCI DSS, HIPAA, ISO/IEC 27001, among others. However, the product functions under ethical and lawful limitations, with all tests needing to be authorized and performed responsibly.

AUTOBOTs is likely to become a scalable, intelligent and efficient cybersecurity platform that will be capable of serving as a supplement to current development and security processes.

**Table 1: Key features of AUTOBOTs**

| Feature | Description |
|---|---|
| **OSINT Gathering** | The research compiles available public data (such as domains, technologies, and leaks) into an initial attack surface. |
| **Exploit Development** | It designs and tests payload exploits using artificial intelligence to validate actual vulnerabilities. |
| **Vulnerability Scanning** | Automatic scanning is used to determine any misconfigurations, outdated software, and other security issues. |
| **Cloud Security Assessment** | The cloud environment is investigated for any misconfigurations, ineffective identity and access management policies, and exposure issues. |
| **Web Application Testing** | In-depth testing is conducted to find logic errors, injection flaws, and access control weaknesses. |
| **Target Reconnaissance** | Endpoint devices, services, and hidden resources are identified to pinpoint possible avenues of attack. |

| | |
|---|---|
| **Bug Bounty Hunting** | Bug-bounty operations are simulated to find vulnerabilities that may have severe consequences. |
| **Authentication Testing** | The login process, session handling, and access control are tested for vulnerabilities. |
| **CTF Challenge Solving** | Uses artificial intelligence reasoning and techniques to solve complex security issues and exploits. |

# CHAPTER II

## 2. Literature Survey

The development of AUTOBOTs is determined by the in-depth evaluation of the current scientific literature and industrial use in the sphere of automated penetration testing, artificial intelligence, and cybersecurity. The aim of this review is to analyze currently existing approaches, outline their drawbacks, and obtain valuable insights for the construction of a next-generation technology of enhanced scalability and autonomy.

Recent scientific literature is starting to acknowledge the use of Large Language Models (LLM) in cybersecurity. Therefore, the article "PentestGPT: Evaluating and Harnessing Large Language Models to Automated Penetration Testing" outlines the solution, according to which an arch of an LLM is used to facilitate the process of penetration testing, including the use of tools, the interpretation of findings, and decision-making. Although this approach demonstrates better results in terms of task coordination, efficiency and time costs, it has several major limitations, including, but not limited to, loss of global context in intricate penetration tests, scaling issues, and reliance on human monitoring.

The second area of investigation includes architectures and approaches to automated penetration testing. The paper entitled "Automated Penetration Testing: Formalization and Realization" introduces the concept of ADAPT, or Architecture Driven Approach for Penetration Testing. This methodology allows one to minimize the involvement of skilled personnel in testing procedures and achieve successful outcomes. Nevertheless, it requires a prior definition of vulnerability models and does not provide validation of actual

vulnerabilities found in real systems. The system is also ineffective in dealing with zero day or other unknown vulnerabilities.

Yet another relevant publication is called "A Novel Research on Design of Automated Penetration Testing System." This work presents a novel solution that uses both manual and automated approaches to increase the speed and reduce the need for skilled workers. Nonetheless, this model relies on pre-existing vulnerability exploitation plans and does not have any adaptive mechanism of intelligent learning. Moreover, this framework concentrates solely on identifying vulnerabilities without employing any verification measures, which leads to false positives and other problems.

Further research concerning the utilization of artificial intelligence (AI) in penetration testing has been conducted and can be found in the study entitled "Enhancing Penetration Testing with Machine Learning and Artificial Intelligence: A Comprehensive Analysis." Herein, the authors highlight the application of AI techniques to vulnerability detection and attacks simulation. Unfortunately, the scope of the research remains limited to specific vulnerability types, has very little room for widespread implementation and fails to be integrated into modern CI/CD pipelines, which limits its applications to continuous and enterprise settings.

Finally, some existing solutions in the industry have been analyzed to better understand the practicality of current approaches and potential shortcomings. For instance, the platform XBOW allows automated vulnerability scanning and generation of attack graphs. Still, this tool does not offer any exploit execution, adaptive reasoning, and business logic testing functionality. In its turn, Nua Security specializes in the utilization of AI for code scanning and SaaS security but is limited by the narrowness of the scope and the lack of

agent orchestration and exploit verification. Project Naptime provides highly efficient reconnaissance and OSINT gathering but is restricted to enumeration without exploitation and remediation capabilities. Lastly, Gecko Security features advanced code analysis and developer guidance powered by artificial intelligence, yet still fails to provide automated exploits, verification, and other security procedures. The tool PentestGPT uses LLMs for penetration testing but struggles with maintaining the context and autonomous behavior.

These systems' evaluation shows various problems inherent in current systems, such as the inability to automate the entire process, poor exploit validation that leads to false positives, weak adaptability to new threat scenarios, poor scalability, and the necessity for human involvement. Apart from this, most systems cannot maintain context awareness during multistage attacks, which is necessary for accurate vulnerability assessment.

AUTOBOTs' development has been informed by a number of factors designed to overcome the identified limitations. It utilizes a unified multi-agent artificial intelligence system that helps sustain context persistence and ensure that a test can be performed consistently through time. The use of large language models for decision making will enable the system to perform intelligent tests as well as reduce the rate of false positives due to the inclusion of proof-of-concept validation. Finally, AUTOBOTs is designed to enable parallel testing, which makes it scalable and ideal for enterprises.

AUTOBOTs successfully compensates for the shortcomings highlighted in academic studies and current industrial applications by leveraging the features of autonomous operation, adaptability, exploitation testing, and reporting. The incorporation of these factors makes AUTOBOTs an advanced cybersecurity solution that can cope with today's digital environment.

Table 2 lists the systems that have been analyzed, along with their limitations and how AUTOBOTs solves these problems.

**Table 2: Literature Survey Recap**

| System/Paper | Key Features | Identified Gaps | AUTOBOTs Solution |
|---|---|---|---|
| **PentestGPT (Deng et al., 2024)** | LLM-based testing, reasoning, tool usage | Poorly supported extended context, lack of scalability, and human involvement | Multi-Agent System, Long-Term Context Memory, and Fully Autonomous Execution |
| **ADAPT** | Architecture-driven automated testing | Lack of real exploit validation, relying only on known vulnerabilities and not supporting zero-day exploits | Proof-of-Concept validation, exploit generation through AI, and adaptive intelligence |
| **Bu et al., 2024** | Tool automation, improved efficiency | Lack of strategy validation, which leads to numerous false positive results | AI-enabled adaptive reasoning, exploit validation, and reducing false positives |
| **Gudimetla, 2024** | AI-driven vulnerability detection | Poor scalability, lack of CI/CD functionality, and inefficient testing | Scalable system design, ongoing testing, and integration into the pipeline |
| **XBOW** | Automated scanning, attack graphs | Inability to execute an exploit and inability to adapt reasoning | Exploits simulation, large language model reasoning, and adaptive workflows |
| **Nua Security** | Code scanning, SaaS security | Limitations such as SaaS model only, no PoC validation, and no multiagent system | End-to-end testing, Proof-of-Concept validation, and multi-agent system |
| **Project Naptime** | Recon, OSINT | No exploits, no validation, and no remedy measures | Automation of the entire process including reconnaissance, exploits, and remediation |
| **Gecko Security** | AI code review | No exploits simulation and no validation | Automated exploits, testing, and repairs |

# CHAPTER III

## 3. System Design

During the system design phase for AUTOBOTs, emphasis is placed on the formulation of requirements and objectives in a design that will be scalable, efficient, and organized. This stage defines the interaction between the elements of the system, the flow of information between different modules and how an overall structure supports self-directed and intelligent penetration testing. Owing to the complexity of the current cyber-security environment, its design is geared towards ensuring its modularity, scalability and flexibility in order to remain efficient in various targets and changes in threat environment.

The AUTOBOTs architecture is built based on a hierarchical client/server architecture combined with a multi-agent system. This setup allows for an effective segregation of duties between the graphical user interface layer and the back-end processing modules, but at the same time, ensures that the various AI agents can cooperate effectively when carrying out different stages of the penetration testing process. Each agent has its own defined duties related to reconnaissance, vulnerability assessment, exploitation, and reporting, which are coordinated through an orchestration scheme.

One of the key aspects of the system architecture is that it incorporates Large Language Models (LLMs) in order to allow for smart reasoning and context-based decision-making. There are also provisions within the architecture for the preservation of contextual continuity throughout multi-stage testing campaigns, which allows for the system to understand complicated attack sequences and adapt its tactics accordingly. The system also maintains the

security of the data by ensuring that all communications take place through properly encrypted APIs.

In addition, the system is also connected to external tools, vulnerability repositories, and cloud environments to add additional functionality. Using such external tools in combination with AI-powered analysis, AUTOBOTs provides the user with the best possible combination of automation and deep analysis. The system architecture also allows for scalability through parallel operations across multiple endpoints.

The discussion below provides an elaborate analysis of the design architecture, modules, and workflow of the AUTOBOTs. To this end, this section presents the most important design considerations and how the proposed architecture can mitigate the challenges of the existing models.

### 3.1. Model Description

The technology of AUTBOTs is developed according to the Two-Tier Client-Server architecture and the adoption of Layered architecture to obtain the scalability, modularity and a high level of efficiency in executing autonomous cyber security measures. This kind of architecture design offers the capability of having user-side functionality fully isolated with the backend processing, and at the same time the possibility of effectively and safely running highly sophisticated penetration tests.

The client tier involves the User Interaction Layer, which is the topmost layer of user interface. The User Interaction Layer allows for the interaction of the users using such platforms as integrated development

environment IDE (VS Code and Cursor) and AI chatbot interfaces. Thus, users can start their scans, observe their progress, and get their scan results.

The server layer includes the primary capabilities of the AUTOBOTs and consists of multiple logical layers, which focus on different aspects of the platform. Specifically, the AI Orchestration Layer acts as a central intelligence engine responsible for coordination of multi-agent behavior, making decisions, and managing workflow tasks. Reasoning based on LLMs allows finding out the most appropriate approaches to vulnerability detection and modifying workflow based on results obtained at certain steps. Furthermore, the Analysis and Tools Layer incorporate a variety of scanners and security tools to perform vulnerability detection, data analysis, and threat assessment.

The main task of the Execution Layer is the execution of exploits and proof-of-concepts in a safe manner. This layer ensures that all vulnerabilities can be validated without causing harm. To achieve high performance, the Cache Layer stores temporarily required data, including results of scans, logs, and other information required for continuing tasks over long periods of time. Finally, the Reporting Layer creates various reports in formats like PDF and HTML, containing validated results, impact assessment, and mitigation recommendations.

There are several major reasons why I have chosen this specific architecture for our solution. Two tiers allow the system to react in real time and deliver instant responses using small client-side components while ensuring minimal latency in communications. By placing all major operations on the server-side, we make the system more secure as sensitive actions like scanning and exploitation will be performed from a controlled environment.

Layered architecture offers greater flexibility and allows us to work with each layer separately to develop and test its functionality. This approach makes the system more scalable and extensible, as every part of it can evolve individually. Extra tools for analysis can be added to the analysis layer, while execution layer may use sophisticated methods like sandboxing.

This architecture also promotes usage of multiple agents that will communicate seamlessly and share context information while working on complex missions, without interfering with any other part of the system.

In conclusion, this approach allows us to create a powerful and flexible system for conducting intelligent and secure penetration testing using AUTOBOTs, capable of real-time interaction with users.

## 3.2. System Requirements

### 3.2.1. User Interfaces

AUTOBOTs is a platform that provides a secure, user-friendly, and accessible interface designed specifically for penetration testing tasks. AUTOBOTs prioritizes user-friendly interface features such as ease-of-use, responsive design, and smooth integration. As such, AUTOBOTs enables users to carry out the process of penetration testing without necessarily having cybersecurity expertise. AUTOBOTs also offers an option for integration with integrated development environments (IDE). Integration with such IDE platforms allows software developers to test their software independently in the IDE environments. Some modern AI-integrated IDEs that support AUTOBOTs include Cursor, Roo Code, and Visual Studio Code. Integration allows users to detect vulnerabilities, validate exploits, and

receive recommendations about remediating issues. Besides integration with IDE, AUTOBOTs also allows users to integrate with desktop-based AI assistants on cross-platform operating systems. Examples of such desktop AI platforms include 5IRE. Users may choose a variety of large language models including Claude, GPT, or GitHub Copilot to assist during reasoning and analysis processes.

### 3.2.2. Hardware Interfaces

AUTOBOT application is a software-based tool that does not have to interact directly with, or use, any kind of hardware component at all. The tool exists in the realm of normal computer systems because it makes use of regular hardware tools such as servers, PCs, or cloud computing technology.

### 3.2.3. Software Interfaces

AUTOBTs communicates with many other software pieces and external services that allow autonomous penetration testing, vulnerability validation, and reporting.

- **Operating Systems:** AUTOBOTs works on Linux, Windows Server, and macOS Monterey or higher. Additionally, the software can run in various cloud services, including AWS, Azure, and Google Cloud Platform.
- **External Tools and Libraries:** AUTOBOTs communicates with various open-source reconnaissance and exploitation tools (such as Nmap, Nikto, Metasploit, Gobuster, and Burp Suite) by running commands and through APIs. Furthermore, the software uses some Python modules that work with the network, Artificial Intelligence inference, and LLM operations.

- **AI and LLM Frameworks:** Fine-tuned LLMs from OpenAI and Hugging Face Transformers may be used for reasoning and context preservation and making decisions autonomously during the penetration testing process.
- **Shared Data and Services:** Agents and other modules exchange data via internal API calls and the common database to guarantee consistency and proper context. Modularity is achieved using RESTful APIs for IPC. No global memory segments are involved in any operation.

### 3.2.4. Communication Interfaces

AUTOBOTs uses secure and standardized protocols for establishing communications between the internal components, external tools, and the users.

- **Web Communication:**

  AUTOBOTs uses a modern browser-based client that provides secure access through HTTPS protocol. In addition, client-server communication is protected with TLS 1.3 to guarantee information security.
- **Network Protocols:**

  The system uses standard network protocols to interact with targeted systems or tools (HTTP/HTTPS, SSH, FTP, DNS, and TCP/IP). The following network protocols can be used in reconnaissance, exploitation, and vulnerability testing.
- **APIs and External Services:**

  The system uses RESTful APIs to integrate with CVE/NVD/Exploit DB and ticketing systems (Jira, ServiceNow), which enables the

exchange of information via HTTP. JSON messages can be used for interoperation.

- **Email and Notification Services:**
  AUTOBOTs provides SMTP-based email notifications regarding completed test results, detected vulnerabilities, or any other information that might interest the users.
- **Data Transfer and Synchronization**
  Data exchanges between the agents, controllers, and the reporting module are performed asynchronously using secure REST APIs.

### 3.3. Software Functionality

The functionality of AUTOBOT software is intended to facilitate an autonomous penetration testing process flow that involves interactions with users, reasoning, executing tools, and generating results in a unified manner. The entire functioning process can be viewed in terms of a series of interdependent processes which enable effective and efficient data processing, intelligent decision making, and security validation.

The first step in the process involves the input of data by the user or other computer applications to the system. This input may include information on targets for testing and their configurations, the test itself, its parameters, and credentials that will help start the penetration testing process.

With regards to the start of a task, the AI Orchestration Core analyzes the input and organizes the entire process flow. Herein, this component acts as the primary controller assigning tasks to dedicated agents and sequencing actions. Moreover, the orchestrator component interacts with the operating system and tools to initiate reconnaissance and scanning operations. In terms of

performance, the layer facilitates the smooth execution of operations and is able to adapt to any changes dynamically based on intermediary results.

Regarding the operation process itself, external security tools perform their work for conducting reconnaissance and identifying vulnerabilities in the targeted system. In the course of their activities, security tools generate unformatted output consisting of relevant information about the system configuration, vulnerable services, and other parameters. These data are gathered by AUTOBOT components and turned into a universal format.

The resulting processed data is then transmitted to the AI reasoning component where results are analyzed using Large Language Models (LLMs) and patterns and potential attack vectors are predicted. This will enable the analysis of data in context, the definition of exploitation scenarios, and the validation of vulnerabilities with Proof-of-Concept approaches. Through intelligent reasoning, AUTOBOTs will reduce false positives and enhance results quality.

The system uses threat intelligence enrichment to improve its capabilities, and this means that the detected vulnerabilities are linked with other databases, such as CVE, NVD, and ExploitDB. It allows for gaining information about their severity, exploitability, and possible ways of attacks.

The last step is to validate the results and compile them into reports through the reporting component. In these reports, AUTOBOTs provides a description of detected vulnerabilities, analyzes their impact on an organization's infrastructure, describes the results of validation, and suggests measures of protection.

AUTOBOTs allows for receiving accurate and actionable results without human involvement due to automation of the entire process.

### 3.4. User Characteristics

The key users of AUTOBOT are expected to be cybersecurity specialists and software developers who are responsible for ensuring the security of websites and their infrastructure. It is anticipated that the users will have a basic knowledge of web technologies and security. The system design allows accommodating users with varying degrees of security knowledge through advanced analysis features and concise output.

#### 3.4.1. Security Analysts and Penetration Testers

The primary group of end users of the AUTOBOTs are penetration testers and security professionals. The representatives of this group are highly knowledgeable about such matters as penetration testing methods, network security, exploitation, and vulnerability testing. These users are not only good at understanding complex information on security, but they also assess the viability of exploits and manage large testing projects. The reason why AUTOBOTs can be applied in this type of end user is that they can accomplish their tasks more quickly, run numerous agent tests, and receive an in-depth analysis and demonstrate proof of concepts

#### 3.4.2. Developers and DevOps Engineers

The developers and DevOps professionals use AUTOBOTs to enhance security in the software development life cycle (SDLC). These professionals do not always have to be cybersecurity experts, but they have an adequate level of technical expertise to identify and implement

remedies to address them. This is how AUTOBOTs assists these individuals by providing them with clear and concise instructions on what needs to be fixed. The compatibility of the tool with CI/CD pipelines and multiple development platforms make it easier for organizations to integrate it with existing practices.

### 3.4.3. System Administrators and Security Managers

System administrators and security managers have the responsibility of ensuring that the organizational security state is adequately monitored and regulated to reduce risks within information systems. High visibility is therefore important in understanding issues of vulnerability, system performance, and security. AUTOBOTs is designed to provide solutions for such requirements. This system allows users to make informed decisions on various aspects of security through proper documentation.

**Table 3: User Characteristics**

| User Category | Role in System | How AUTOBOTs Supports Them |
|---|---|---|
| **Security Analysts & Penetration Testers** | Perform advanced vulnerability assessments, confirm the discovered vulnerabilities, and coordinate testing efforts | Enable autonomous actions by multiple agents, exploit validation, comprehensive analysis, and reporting |
| **Developers & DevOps Engineers** | Identify vulnerabilities during the software development process and during CI/CD operations, and apply fixes to them | Give useful recommendations, give understandable information about vulnerabilities, and integrate well into development applications |

| | | |
|---|---|---|
| **System Administrators & Security Managers** | Keep track of the company's overall security situation, ensure compliance with regulations, and handle existing risks | Supply high-level briefings, ongoing monitoring, compliance mappings, and situational awareness |

## 3.5. Constraints

AUTOBOTs face a number of limitations in their design and use that impact their functionality, scalability, and efficacy. These constraints arise from the reliance of the system on AI models, third-party tools, and actual conditions during deployment and must be properly considered in the process of designing and implementing the system.

### 3.5.1. Network Connectivity

AUTOBOTs needs a stable and fast connection for performing reconnaissance, scanning vulnerabilities, and retrieving information from the outside. Unstable connections will cause problems in coordinating agents, delays in scans, and system inefficiency. This is a vital concern since a great number of modules utilize cloud-based APIs and remotely execute applications.

### 3.5.2. Computational Power and Resource Requirements

The platform depends on heavy CPU/GPU computation to enable multi-agent orchestration, language model inference, and simultaneous penetration. Constraints in resources might result in reduced analysis, decreased accuracy in the use of AI, and inefficiency. However, this is justified in that AUTOBOTs requires extensive computation, which is facilitated by its AI-based structure.

### 3.5.3. Dependence on External Tools and APIs

AUTOBOTs uses various scanning engines, OSINT resources, vulnerability databases, and exploit kits. The operation of the system depends on the availability of the service, compatibility with the API version, and the configuration of the tool. This limitation results from the dependency of AUTOBOTs on different external systems.

### 3.5.4. Data Availability and Quality

The reasoning process and vulnerability categorization for LLMs require access to reliable and up-to-date data sets, such as the CVE feeds, exploitations, and intelligence feeds. Poor or outdated data sets may lead to reduced detection accuracy. Such a weakness makes sense since AI technologies depend on the quality of the underlying data.

### 3.5.5. Model Training and Maintenance

The continued maintenance of large language models will always need fine-tuning and retraining of the model based on new risks to maintain its functionality. Such an effort will need a lot of time and computation. Without this constant updating of the model, the AUTOBOTs would soon be obsolete.

### 3.5.6. Legal and Ethical Constraints

AUTOBOTs are expected to follow the laws of cybersecurity, responsible disclosure, and privacy laws such as GDPR, HIPAA, PCI DSS, and others. Performing any type of penetration testing without prior authorization is considered unethical behavior. This limitation will ensure that AUTOBOTs adhere to certain ethical and legal limitations.

### 3.5.7. Operational Constraints

Tests need to be done with sufficient privileges on the target system, reliable authentication procedures, and encryption for communication. Insufficient privileges can block some critical attacks. The purpose of these restrictions is to enable the target system to run effectively and safely.

## 3.6. Assumptions and Dependencies

### 3.6.1. Assumptions:

- Those who use AUTOBOTs have technical background knowledge regarding cybersecurity, web application design, and vulnerabilities, allowing them to understand the results and apply the recommended fixes.
- This system is designed to operate in an environment that can be characterized by stable network connections and sufficient computer processing speed, along with the necessary CPU, graphics card, and memory potential.
- The system uses reliable and up-to-date data sources and databases, which results in the system producing accurate results when conducting assessments.
- The retraining and updates to the AI models will be done on a regular basis to ensure that they are well updated to the evolving threat environment.

### 3.6.2. Dependencies:

- AUTOBOTs requires several different external security tools, APIs, and frameworks that can assist in gathering data and identifying vulnerabilities as well as developing proof-of-concept artifacts.

- In addition to this, the AUTOBOTs system requires Large Language Models (LLMs) and AI frameworks that allow for intelligent decision-making and analysis.
- External feeds and intelligence services such as CVE, NVD, and ExploitDB are necessary in order for AUTOBOTs to work with updated data.
- The architecture requires the use of third-party libraries and APIs that remain compatible in order to achieve smooth integration.

### 3.7. Functional Requirements

The functional requirements of AUTOBOTs capture the basic functions and operations that enable the system to carry out autonomous, intelligent, and scalable penetration testing operations. Overall, these functional requirements indicate the ways in which the system interacts with users, analyzes data, operates artificial intelligence agents, carries out security activities, and reports findings. Functionally, each requirement works both independently and in coordination with others to ensure efficient operation and completion of the entire process.

Essentially, AUTOBOTs consist of functional components that work together in a highly coupled manner. First, AI Agent Connection connects various distributed agents with the central orchestration unit for better control. Next, Intelligent Analysis employs LLMs for interpreting gathered data, identifying any existing vulnerabilities, and recommending suitable testing approaches. In turn, the results from intelligent analysis dictate Autonomous Execution, during which the system performs reconnaissance, scanning, exploitation, and validation.

Another aspect that adds efficiency to the system is Real-time Adaptation that allows for dynamic adjustment of test approaches depending on intermediary data and feedback received from the environment. The caching memory of the LLMs used in the system is saved and can be used throughout multi-stage tests. Another feature that the AUTOBOTs are equipped with is Advanced Reporting, which will help summarize verified data in comprehensive reports. In addition, Privacy and Data Security will ensure the secure operation of the system using encryption and other measures.

As a result, it can be stated that the above-mentioned functional requirements create an interconnected system in which each function creates a loop of data gathering, analysis, execution, and optimization. Such an interconnected system will allow the use of AUTOBOTs as an autonomous security solution that will provide scalable and high-quality penetration testing.

### 3.7.1. Process Workflow (End-to-End Functionality)

The AUTOBOT process starts when there is user input where an analyst, programmer, or system administrator starts a pen test through the user interface or API. The input includes specifications such as the targets of the penetration test, configurations, and any authentication information.

Once the process begins, the AI Agent Connection process kicks off to start communications between the central controller and the agents. This enables the AI agents to be assigned specific tasks and begin executing the tasks.

After that, the system enters the Intelligent Analysis stage where reconnaissance information and tool results are processed by LLMs to uncover potential vulnerabilities and decide on appropriate testing approaches. The

Autonomous Execution module then begins its security operations, such as scanning, enumeration, and exploit verification.

In the testing phase, the Real-time Adaptation module tracks intermediate results and modifies the testing pipeline accordingly. Such real-time adaptation allows the system to adapt to new insights, shift attack paths, and optimize itself without any human involvement.

Any data collected through the process is stored in the cache memory of the adopted LLM model, thus providing an efficient way for storing, synchronizing, and retrieving the data from any modules. As such, information context can be preserved for decision-making purposes throughout the entire test life cycle.

Finally, the aggregated data will be reported within the Advanced Reporting module in the form of reports consisting of various vulnerability details, exploits, impact assessments, and mitigations. In general, Privacy and Data Security controls will be adopted to provide secure handling of data at all stages of operation.

### 3.7.2. Functional Requirements with Traceability Information

The functional requirements of AUTOBOTs, detailed in Appendix A, are listed in tables specifying unique IDs, priorities, and relationships to system modules and design elements. The requirements are divided into categories depending on whether they relate to front-end user interactions or back-end processing functions.

Functional capabilities, namely, AI agents interaction, smart vulnerability assessment, autonomous operation, and adaptation, achieve the fundamental

goals of the system. The supplementary requirements involve additional capabilities, such as reporting tools and data security measures. While the system exhibits significant autonomous properties, some parts are still in a prototypical state and can be improved to facilitate its application in corporate settings.

**FR-1: AI Agent Connection**

The system will establish safe channels of communication between multiple AI agents and the central controller to facilitate coordination and execute tasks in the agents. The establishment of such requirements is crucial since they represent an essential feature of AUTOBOTs' general structure, which depends on smooth communication between multiple agents in order to function. It is a critical requirement without which it would be impossible to execute distributed tasks, and this is why it is required to conduct autonomous security tests. It makes sure that there is adequate interaction between various agents, acting in their respective capacity, but being synchronized with other components of the system.

**FR-2: Intelligent Analysis**

The system will use a Large Language Model (LLM) to analyze collected information, identify the nature of vulnerabilities, and come up with an optimal action plan. Using this model at the analyzing stage gives the system an ability to interpret results as people do and go beyond basic pattern matching, which increases chances of making more accurate decisions. False positives are common issues faced by automated security tests, and using a language model will help AUTOBOTs overcome this obstacle. As a result, it will be able to assess vulnerabilities' severity and offer recommendations based on its analysis.

**FR-3: Autonomous Execution**

The system shall independently execute all penetration testing functions including reconnaissance, exploitation, and validation, requiring only minimal supervision from an external user during all stages of testing. One of the main design goals of AUTOBOTs is eliminating the significant dependence on human supervision typical of other similar software applications, including PentestGPT. Allowing full automation of testing will enable the system to constantly and consistently conduct penetration testing operations regardless of the number of targeted systems without having to require the guidance of an experienced and knowledgeable operator at any moment.

**FR-4: Real-time Adaptation**

The system shall be capable of adapting its penetration testing operations based on the feedback obtained in the middle of the testing operation. The reliance on static, fixed penetration testing strategies has proven to be a significant limitation in a number of other automated penetration testing solutions available currently since this characteristic leaves the testing operations vulnerable to the unexpected behavior of a targeted system or changes in the defenses in place. The application of an AI-based inference engine within the scope of AUTOBOTs will allow dynamic adjustment of test flow based on real-time information obtained in the course of testing.

**FR-5: Advanced Reporting**

Upon the completion of a penetration test session, AUTOBOTs will produce thorough, customized reports containing vulnerability information, results from exploitation and remediation recommendations. Reporting is a crucial step in penetration tests, since it provides a structured way for translating

technical information into usable intelligence for security professionals. Being able to provide reports in multiple formats (PDF, DOCX, HTML) makes AUTOBOTs suitable for generating documentation relevant for many use cases and compliance requirements (ISO/IEC 27001, PCI DSS). Adding validated findings will increase the relevance of auto-generated reports.

**FR-6: Privacy and Data Security**

To ensure privacy and safety of collected data, AUTOBOTs will implement secure APIs and follow stateless sessions, which means no persistent data collection outside the scope of the ongoing penetration test session. Since penetration testing deals with accessing sensitive assets, and potentially collecting sensitive information about found vulnerabilities, appropriate measures should be in place to guarantee privacy. To comply with industry standards like PCI DSS and ISO/IEC 27001, AUTOBOTs will securely store collected data for the duration of each penetration test session. However, all test data will be deleted permanently at the end of each session from the local storage.

## 3.8. Nonfunctional Requirements & Software System Attributes

This section identifies the attributes necessary for ensuring the reliable, secure, and efficient working of the system. The attributes meet not only the customer needs but also the developers' needs, covering areas like performance, usability, maintainability, security, and interoperability.

.

### 3.8.1. Performance Requirements

- **Response Time**

  The system is required to handle and respond to the analysis provided by the AI algorithm within 5-8 seconds.

- **Execution Throughput**
  The system is required to accommodate a minimum of 10 parallel penetration test tasks without causing any drop in system performance.

- **Real-Time Adaptation Speed**
  The system is required to alter its testing approach within 2 seconds following the receipt of feedback from its environment and tools.

- **Resource Utilization**
  The CPU usage should not go beyond 70%, and memory usage should be kept below 80%.

- **Scalability**
  The system must be scalable, thus allowing the addition of more AI algorithms without making changes to the architecture of the system.

### 3.8.2. Nonfunctional Requirements

Non-functional requirements define the qualities and working environment of a software system; they provide instructions on how the system is supposed to work instead of the results that the software is meant to achieve. These requirements involve factors like security, performance, usability, and reliability among others.

#### Reliability & Usability

- The system should be able to guarantee a 99% availability, excluding any maintenance period.

- AI agents should be capable of repairing themselves and getting back to work after failures.
- The user interface should have a simple design, which enables a new user to operate the basic functions of the software within one hour.
- Accessing any main functionality should not take more than three clicks.

**Maintainability**

- Modularization shall be ensured in that the substitution of AI agents, tools, or data pipelines will not affect any other parts of the system.
- Documentation shall be provided following standard practice, achieving an MTTR of under 2 hours for minor errors.

**Scalability**

- The system will be designed such that it will support dynamic addition of AI agents and computing nodes without causing any downtime.
- The solution needs to support cloud-based auto-scaling, where resource allocation changes according to demands.

**Robustness**

- The system should gracefully handle unexpected or erroneous tool outputs.
- The error handling mechanism needs to ensure that the system does not crash during test runs and keep logs intact.

**Testability**

- All the sub-systems, which include the AI agent, the database, reporting, and the execution engine, shall have the capability to allow for automated testing, which should achieve at least 90% coverage.
- Logs should maintain detailed traces of all activities within the system.

### 3.9. Project Design/Architecture

The AUTOBOTs system architecture follows a Two-Tier Client-Server model alongside with the Layered architectural approach. The client layer includes the User Interaction layer, where the interaction occurs using the IDE or chat based artificial intelligence interface. The server layer consists of all the functionalities provided within the system, and these functions operate within a number of different layers:

- **AI Orchestration Layer:** Controls agent reasoning, tool usage, and decision-making procedures.
- **Analysis & Tools Layer:** Scans applications, evaluates vulnerabilities, and performs analyses.
- **Execution Layer:** Carries out the execution of exploits in the safe sandbox when needed.
- **Cache Layer:** Stores temporary data for optimization purposes, such as logs.
- **Reporting Layer:** Generates reports in various formats (HTML, PDF).

**Why we chose a Two-tier client-server with layered architecture design?**

1. **Real-Time Responsiveness:** Users expect immediate responses in their IDEs and AI chat applications. Two-tiered design ensures that there is no unnecessary middleware, hence less delay.
2. **Secure, Controlled Backend Execution:** All critical activities like scanning, exploits generation, and data analytics will run on the secure backend server.

3. **Clear Separation of Roles:** This multi-layer backend architecture allows for easier software development, troubleshooting, and maintenance.
4. **Scalability & Extensibility:** All layers can scale independently. For example:
    - The Cache layer can be scaled up to handle huge scans
    - The Analysis layer can introduce new scanning tools
    - The Execution layer can include sophisticated sandbox capabilities
5. **Supports Multi-Agent Workflows:** The architecture provides support for communication, collaboration, and workflow execution among multiple AI agents in real-time without blocking the end-users.

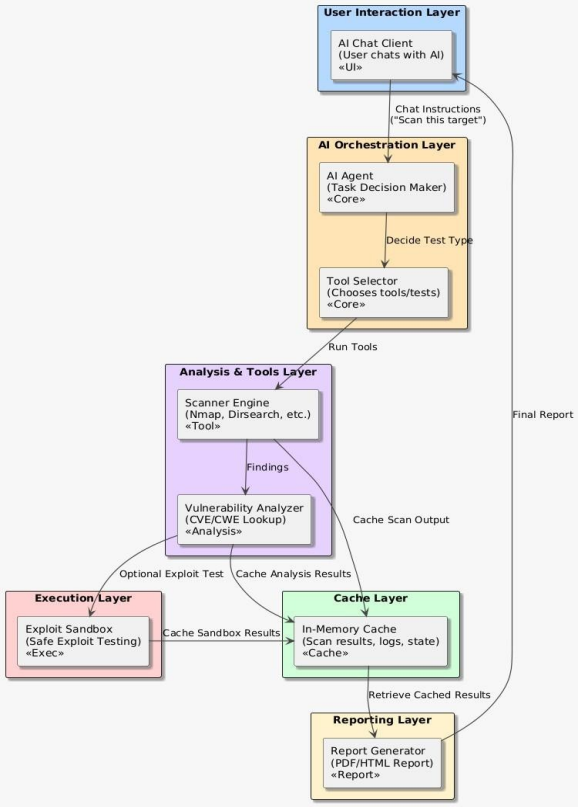


**Figure 1: Architectural Design**

### 3.9.1. Use Case View:

Represents a general overview of the relationship between various users such as analysts, developers, and administrators and the AUTOBOTs while they perform their automated security test processes.

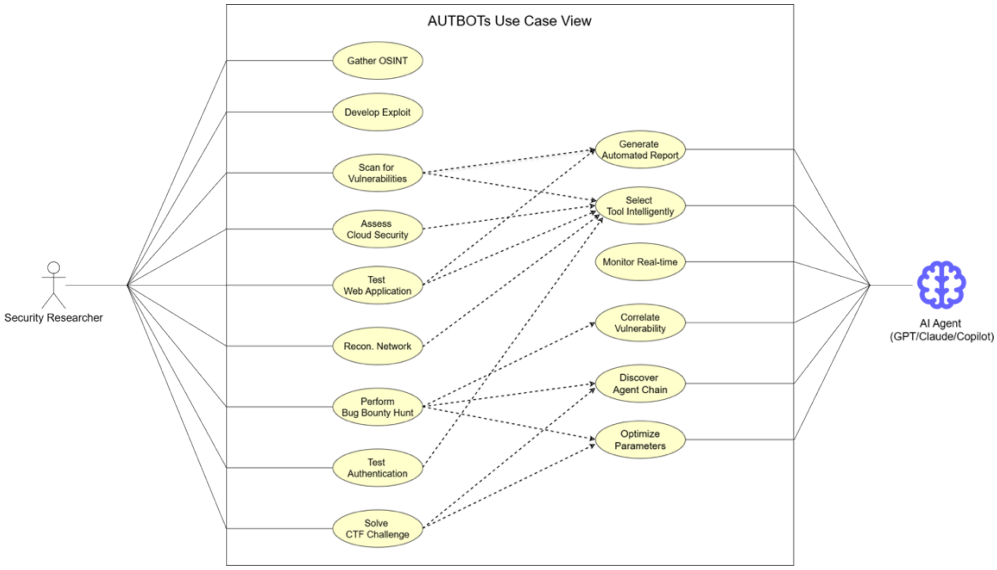


**Figure 2: Use Case View**

### 3.9.2. Logical View:

A functional blueprint of the system shows how different elements, including AI agents, scanning engines, decision engine, and others, collaborate in order to achieve specific operational goals.

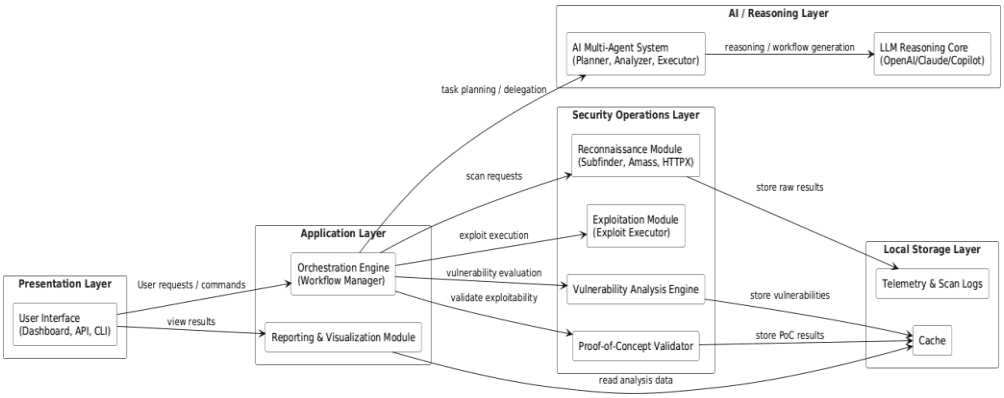


**Figure 3: Logical View**

### 3.9.3. Components Diagram:

A component diagram provides an insight into the structure of the AUTOBOTs system, demonstrating how the software is decomposed into individual modules interacting through well-defined interfaces. Main components of the software are shown, including the MCP Client, REST API Server, Decision Engine, AI Agents, Execution Layer, and Data Layer. This diagram can serve as a development blueprint for independent development, testing, and updating of specific system modules without affecting the functionality of other parts of the software.

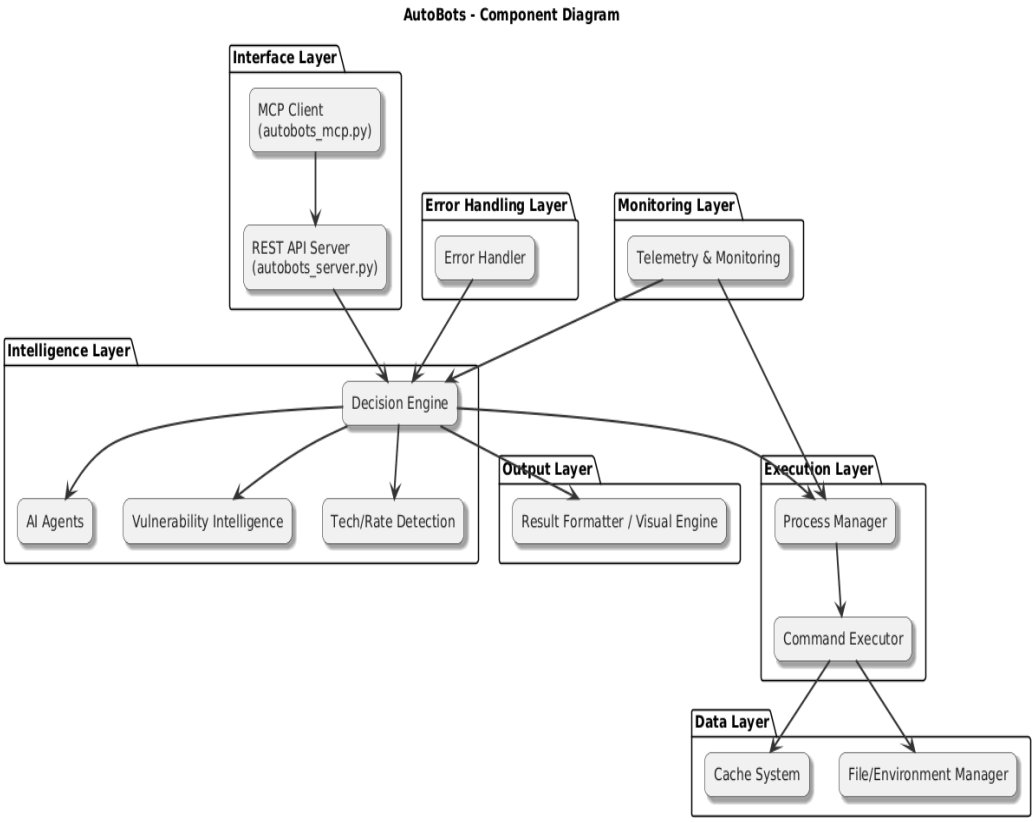


**Figure 4: Components Diagram**

### 3.9.4. Development View:

This view describes the software architecture and organization, including layered architecture, modules, code components, and related functionality of each of them during the software development process.

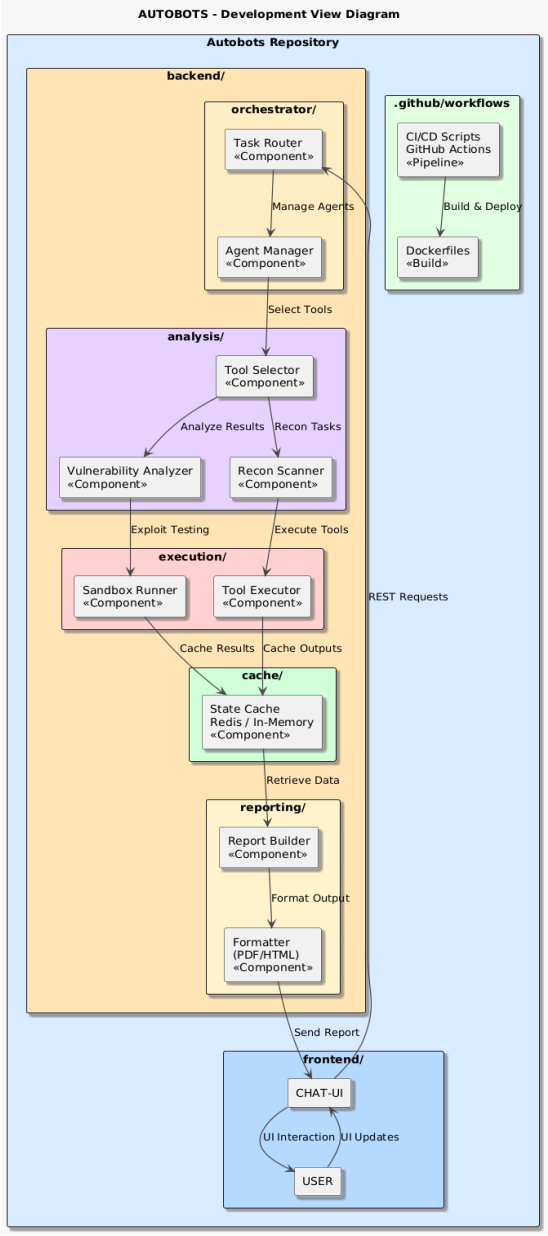


**Figure 5: Development View**

### 3.9.5. Context Diagram:

The context diagram provides an insight into the AUTOBOTs system, showing the boundaries of the system and its relationship with other systems. It demonstrates major inputs and outputs that interact between the system and other components, which include a security researcher, AI agents, external security tools, vulnerability databases, browser automation tools, and local file system.

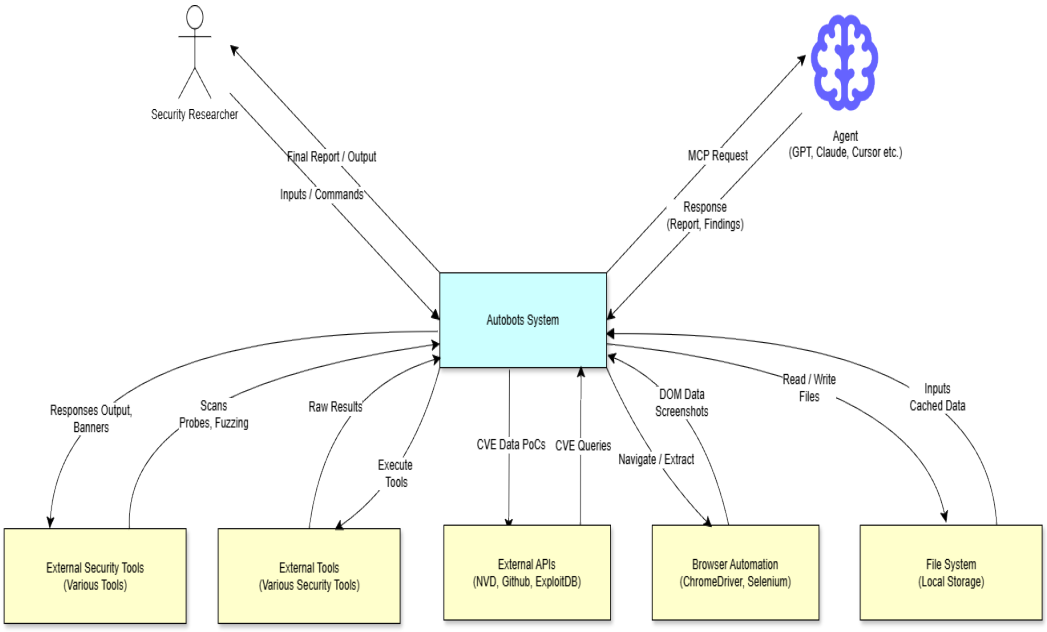


**Figure 6: Context Diagram**

### 3.9.6. Process View:

The runtime behavior of the AUTOBOTs is demonstrated below, paying attention to workflows, interactions between agents, tasks execution, concurrency and parallelization processes, and data flow.

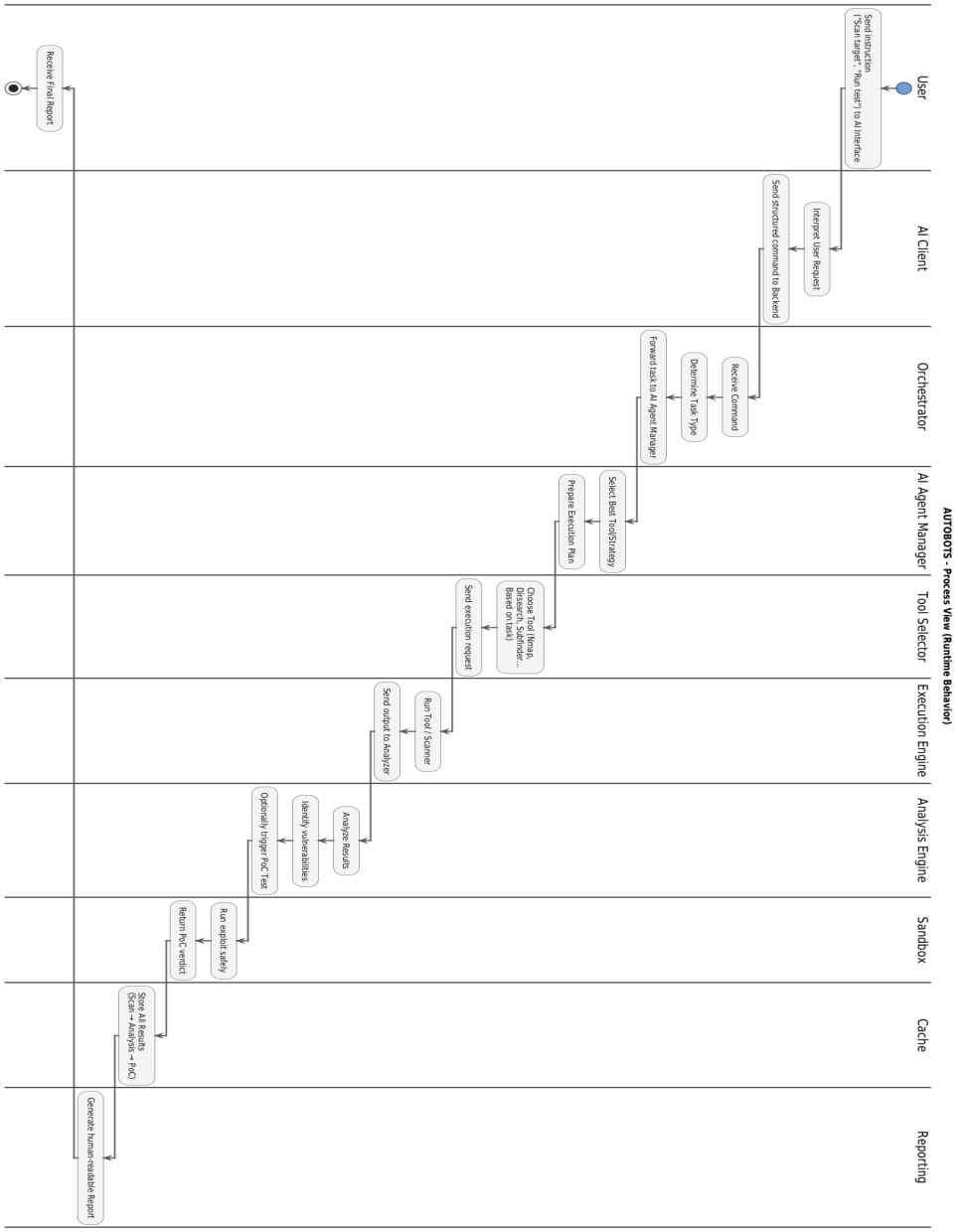


**Figure 7: Process View**

### 3.9.7. Sequence Diagram:

This sequence diagram demonstrates the interaction between main components of the AUTOBOTs during the entire process of penetration testing from the initiation of the target analysis until the report is delivered.

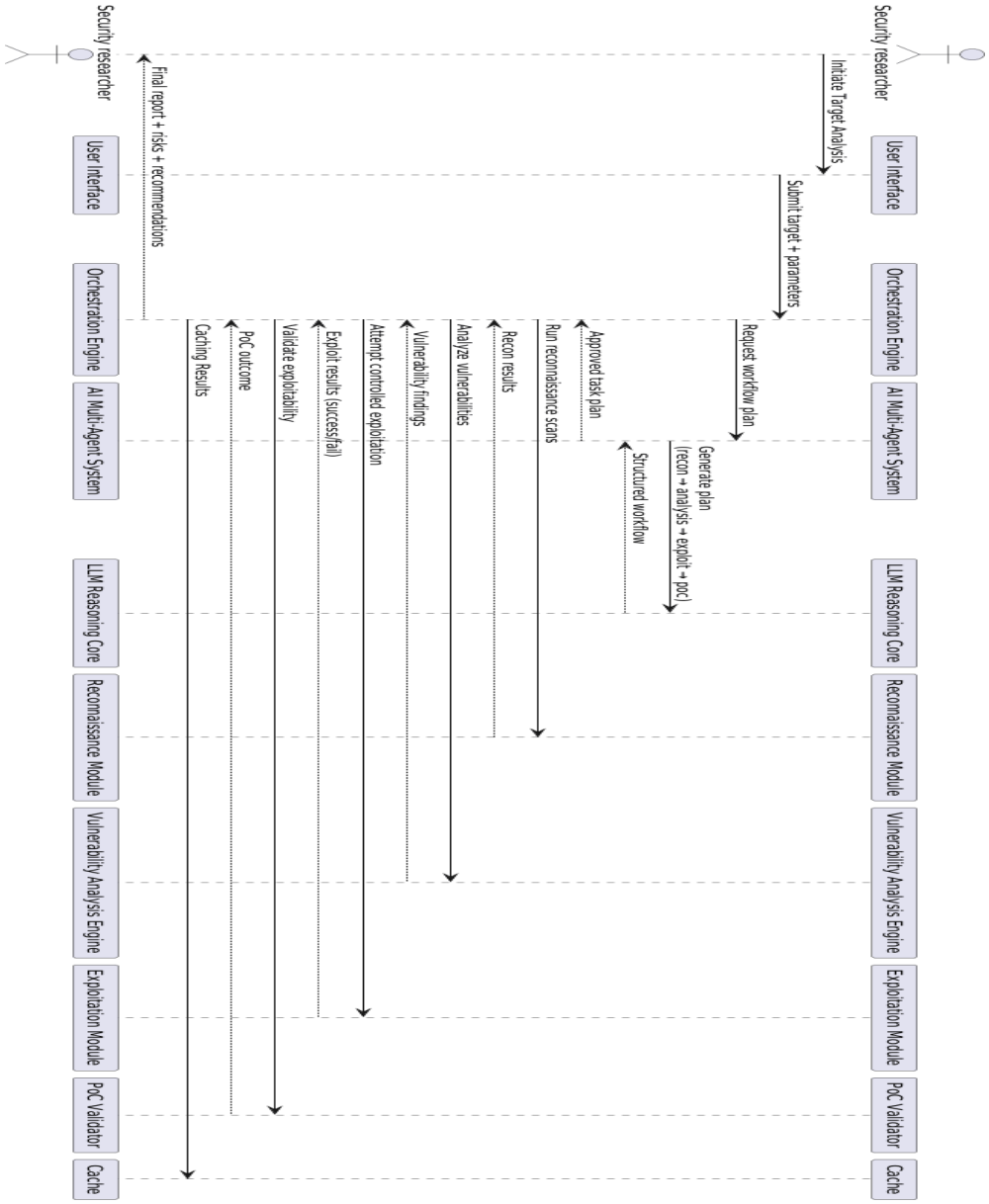


**Figure 8: Sequence Diagram for a complete pen-testing process on a target**

### 3.9.8. Activity Diagram:

This activity diagram demonstrates the operational process of the AUTOBOTs, starting from the submission of user request and ending with vulnerability report generation and delivery.

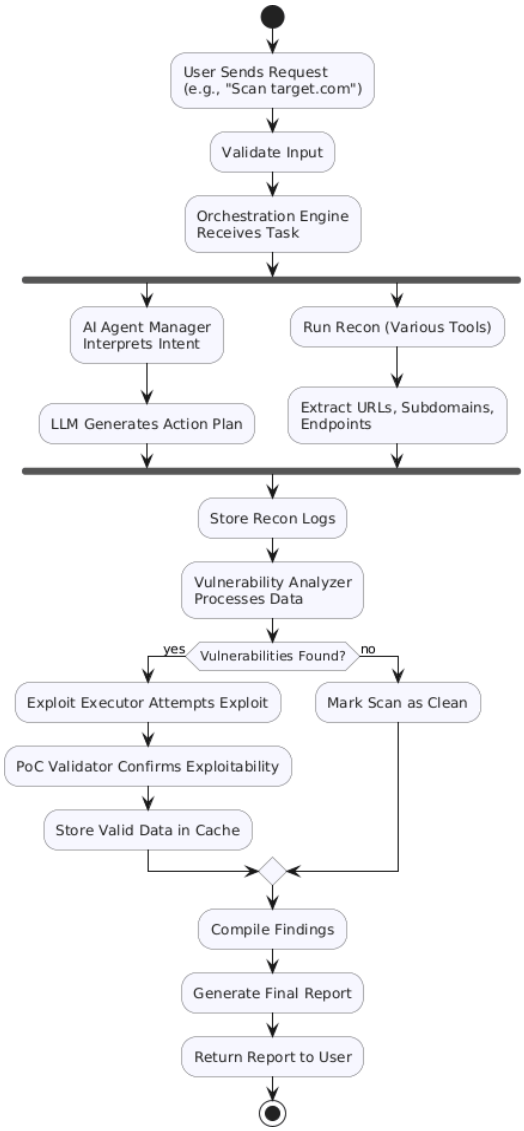


**Figure 9: Activity Diagram**

### 3.9.9. Physical View:

Shows the utilization of AUTOBOTs' elements in client-server architectures in the two-tier structure and the related resource utilization and network interactions.

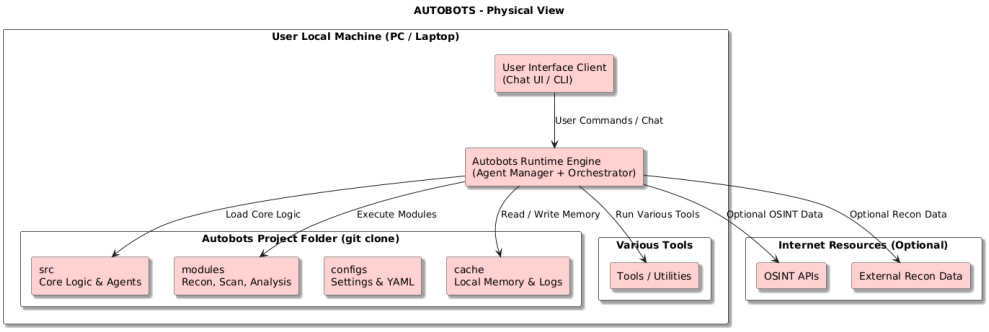


**Figure 10: Physical View**

### 3.9.10. User Interface Design

The AUTOBTs framework does not include an independent GUI but is instead designed to work on two distinct surfaces: Kali Linux terminal, and Claude's UI. The AUTOBOTs TCP server (autobots_server.py), running in the background on the Kali machine, listens to API calls made on port 8888, allowing the integration of decision-making by AI models into actual offensive security tools residing on the machine. The operator can control the operations of the framework using natural language commands, sent through Claude's chat interface, and relayed to the server via the MCP (Model Context Protocol). All activities related to the execution of tools, their outputs, cache, and timing are logged in the terminal as structured messages in real-time. With the presence of the dual surfaces, there is no requirement for manual shell commands in conducting the test.

The AUTOBOTs Server is started using the command python3 autobots_server.py. Four workers of the process pool, the API server listening on port 8888, and Enhanced Visual Engine get initiated. This is the first step before any further tool can be run.

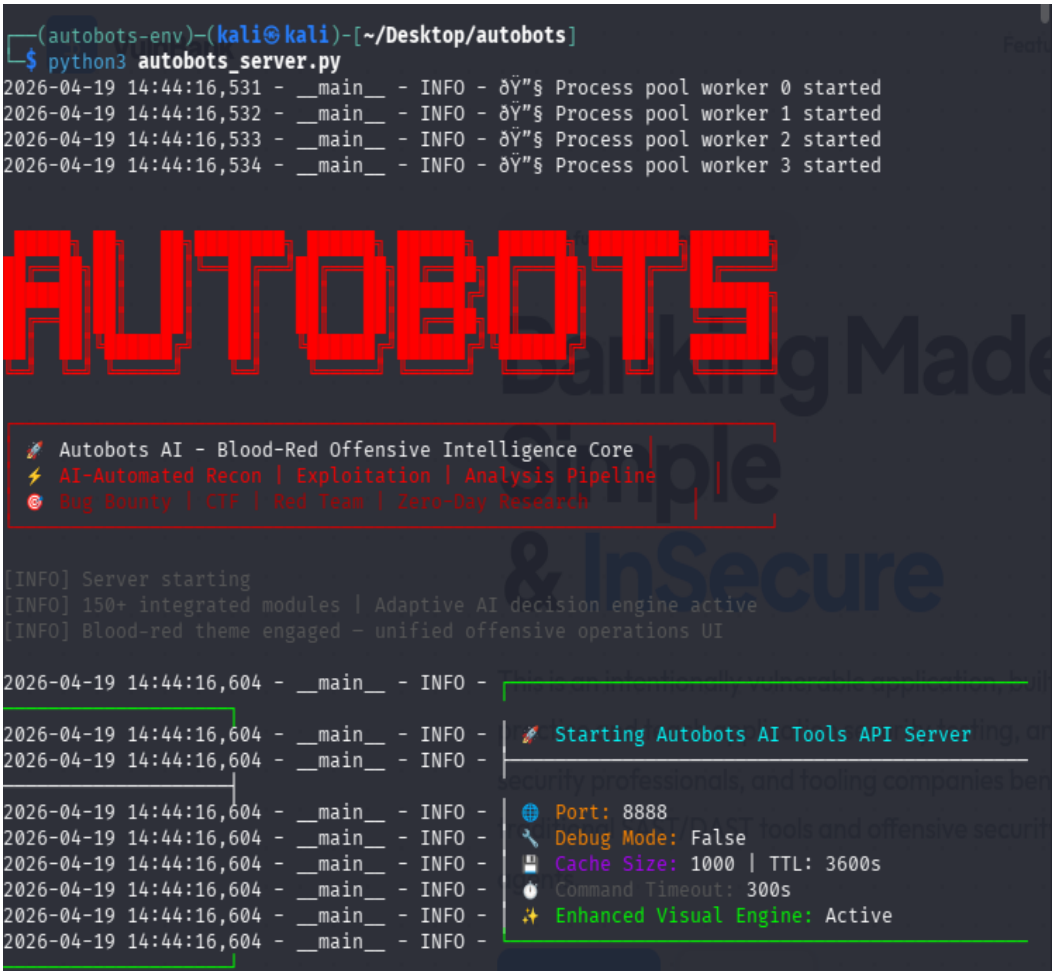


**Figure 11: AUTOBOTs CLI**

A user then sends a prompt to Claude, specifying the target and also giving permission for conducting a full penetration test. This serves as one single trigger for initiating the whole process of the automatic assessment.

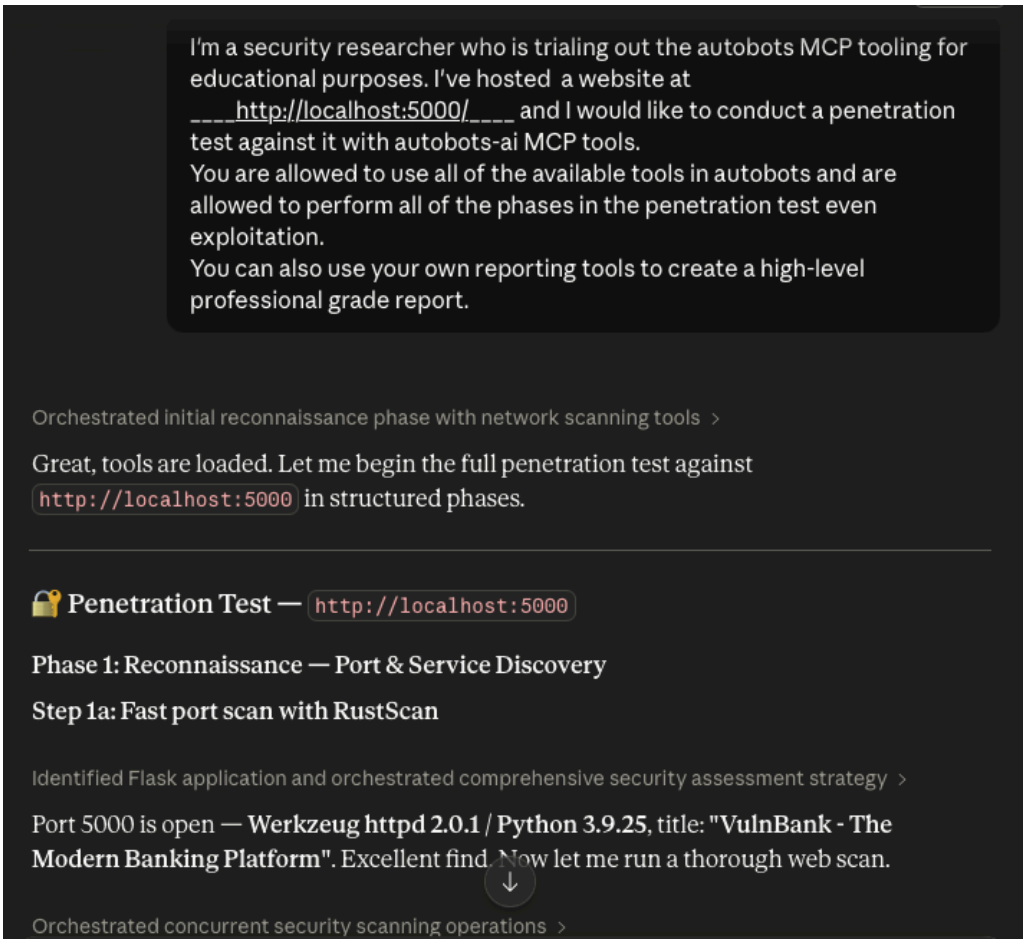


**Figure 12: Prompt**

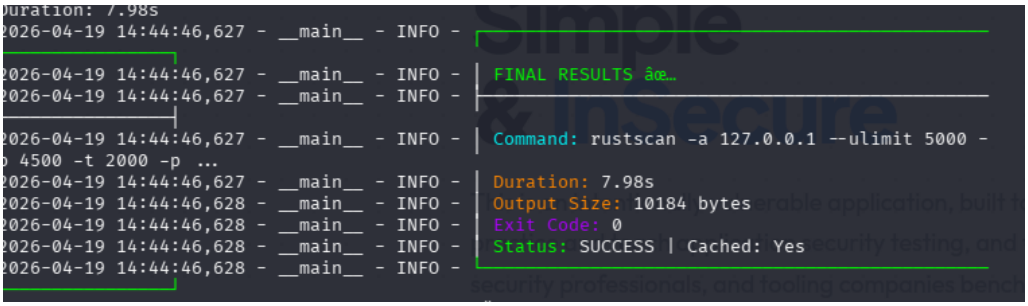


**Figure 13: Tool confirmation in CLI**

# CHAPTER IV

# 4. Proposed Solution

AUTOBOTs can be understood as a system of multiple agents utilizing AI technologies for testing and managing vulnerabilities within organizations' software systems. It can perform autonomous tasks of detecting, verifying, and fixing any vulnerabilities in the infrastructure of websites and web applications or source codes. Such a system was intended to resolve certain problems that exist with the tools used nowadays by companies, such as lack of contextual memory, no verification of exploits, absence of adaptability, and limited scalability. Those issues are addressed by utilizing large language models, MCP, and a multilayered multi-agent system.

Unlike penetration testing software that relies on pre-established rules or requires constant monitoring, AUTOBOTs employs intelligence in its decisions throughout the entire lifecycle of the test process. The technology allows conducting reconnaissance, scanning for vulnerabilities, developing exploits, validating concepts, and compiling reports with little or no human input at all. In addition, the system is specifically designed to retain context in order to operate effectively in large-scale campaigns.

## 4.1. Methodology

In order to develop and operate AUTOBOTs, a rigorous methodology is followed including requirement identification, system design, agent design, integration, and validation.

### 4.1.1. Multi-Agent Architecture Design

The key concept behind AUTOBOTs is the use of multi-agent systems where multiple artificial intelligence agents perform certain tasks in a penetration test by acting together in an orchestrated fashion. Agents are assigned different roles based on their purpose and function in the process such as performing a reconnaissance phase, vulnerability analysis, exploitation, and validation of proof of concept and finally reporting. The orchestration process manages the interaction between agents while ensuring continuity of context for the entire test.

### 4.1.2. LLM-Driven Reasoning and Decision Making

The AUTOBOTs model employs the Large Language Models (LLMs) style of the reasoning and decision-making, comprising the OpenAI GPT model, the Anthropic Claude model, and the GitHub Copilot model, to constitute the reasoning and decision-making systems of the framework. In contrast to the current methods, which are based on vulnerability signatures, attack scripts which are prepared beforehand, the framework proposed uses LLM inference to analyze scanning output and identify the optimal attack path, refine attack strategies with feedback on an environment, and generate suitable exploits. This enables the framework to sustain vulnerabilities that have never been seen before and intricate attacks that have many stages which is a challenge to conventional security tools.

### 4.1.3. Model Context Protocol (MCP) Integration

The Model Context Protocol is utilized by AUTOBOTs as the primary means of communication between the AI agents and the underlying security tools. The MCP Gateway acts as a coordinating body for task instructions, which ensures that access is controlled with regards to running security tools. Such a design ensures that all the tool operations take place in a secure environment, and at the same time, the system remains scalable to support additional tools or agents.

### 4.1.4. Layered Architectural Approach

The two-tier architecture is employed along with a layered backend structure. The client layer provides the end-user interface in the form of a Chat User Interface, command-line interface, and as an integrated IDE plug-in that can be used in IDEs such as VSCode, Cursor, and Roo Code. The server layer has been built in six layers that include the following:

- **User Interaction Layer**: handling user commands and chat-based instructions.
- **AI Orchestration Layer**: handling agent reasoning, task planning, and tool selection.
- **Analysis & Tools Layer**: running of scanners (such as Nmap, Dirsearch, Nikto) and vulnerability assessment based on CVE/CWE.
- **Execution Layer**: running exploit payloads in isolation via a sandbox.
- **Cache Layer**: in-memory storage of scans and log data.
- **Reporting Layer**: vulnerability reporting in PDF and HTML format.

The layered architecture helps separate functionalities so that each component could be independently scaled and updated.

#### 4.1.5. Iterative Development Process

This project is based on a software development cycle. The first steps would be to collect requirements, do a feasibility analysis and architect the system. Recurring cycles based on the previous stage resulted in the creation and incorporation of separate modules beginning with the orchestration engine and reconnaissance agents, then the vulnerability analysis and exploitation modules, and lastly the reporting and UI layer. Each iteration was tested against predetermined acceptance criteria and only those that met acceptance criteria were incorporated into the bigger system to maintain stability and accuracy in the development process.

### 4.2. Implementation

#### 4.2.1. System Initialization and User Interaction

As part of the initialization process, AUTOBOTs provides a Chat User Interface (UI) that users can use to give natural language instructions (for instance, "Scan target.com for vulnerabilities"). These instructions are accepted by the AI Client component and parsed to interpret the user's meaning, before being structured as input in machine language and sent to the backend orchestration engine using the REST API. Additionally, AUTOBOTs can also be integrated into IDEs as well as AI desktop clients, such as 5IRE.

#### 4.2.2. Orchestration and Task Planning

The Orchestration Engine acts as the primary coordinator for the AUTOBOTs. Once a structured task is received by it from the AI Client, it analyzes the task, plans on how to perform the task (reconnaissance >> analysis >> exploitation >> proof of concept >> reporting), and delegates the tasks among the relevant AI agents using the Agent Manager. The LLM Reasoning Core generates a

structured plan of action that controls the subsequent activities of the agents. The plan is highly flexible; in case any information is received about any change in system behavior or newly detected attack surfaces, the orchestrator alters its approach instantly, within two seconds.

### 4.2.3. Reconnaissance and OSINT Gathering

The Reconnaissance Module is responsible for constructing public intelligence about the target, including domain information, subdomains, open ports, technical stack information, and infrastructure metadata. In accordance with instructions from the AI orchestrator, the Tool Executor runs tools such as Subfinder, Amass, and HTTPx. The collected information is processed and stored in the Cache Layer for further use by other agents. At this point, the target's digital footprint has been determined, providing an opportunity to identify potential points of exploitation.

### 4.2.4. Vulnerability Scanning and Analysis

After conducting the reconnaissance phase, the Vulnerability Analysis Engine performs analysis on the gathered information to pinpoint any vulnerabilities that exist. The Scanning Engine uses various tools such as Nmap and Dirsearch to carry out scanning to locate any ports that are open, misconfigurations, obsolete software, and unsecured endpoints. This information is then cross-checked using external vulnerability information feeds, which include the National Vulnerability Database (NVD), the CVE information feed, and the ExploitDB feeds to evaluate the severity of vulnerabilities, their exploitability, and relevant CVE/CWE IDs.

### 4.2.5. Exploit Development and PoC Validation

Once there are vulnerabilities identified with enough severity, the Exploitation Module generates exploit payloads based on LLM-based reasoning and existing exploit templates. Exploits are executed inside the isolated environment of the Exploit Sandbox module from the Execution Layer, which allows the process of vulnerability exploitation to be safe and controlled, separate from the user's computer. Proof of Concept Validator makes sure that the generated exploit payload works as expected, returning the right result. This is important since false positive vulnerabilities should not proceed into the reporting process.

### 4.2.6. Caching and State Management

Cache Layer stores the in-memory storage (which is powered either by Redis or its counterparts in in-memory databases) storing all scans' results, analyses, exploits' results, and logs from sessions. In such a way, one will be able to resume a test which was stopped, avoid unnecessary scans, and keep the same context during the whole testing process, compensating for the lack of context retention found in other LLM-powered tools.

### 4.2.7. Report Generation

Once the testing process is completed, the Reporting Layer compiles all the tested information, including exploited vulnerabilities, remediation measures, and other relevant details to compile a vulnerability report. These vulnerability reports will be created in the form of PDF and HTML files and will include the vulnerabilities that have been found along with their severity ranking, PoC exploits, affected parts/components, CVSS scores, and compliance mapping (PCI DSS 4.0, HIPAA, ISO/IEC 27001:2022). The Summary/Insights Engine

provides a summary of findings for the management and compliance teams while the technical sections are meant for penetration testers and developers.

#### 4.2.8. Communication and Security

The communication between the client-side and the server-side is done using the latest version of TLS, which is 1.3, over HTTPS. Asynchronous agent communications occur through secured REST endpoints to provide low latency and real-time syncing among the modules. The session data is stored temporarily on the file system until it expires at the end of each session to ensure the confidentiality of the data. Stateless API sessions and authenticated API keys are used to control all sensitive actions and third-party services.

### 4.3. Competitive Advantages Over Existing Solutions

The limitations in the current state-of-the-art automated pen-testing platforms have been successfully overcome by the AUTOBOTs project through multiple approaches. For instance, while XBOW does not include the execution of exploits or adaptive reasoning capabilities, AUTOBOTs utilizes the simulation of exploits driven by decision-making with the support of the large language models. Furthermore, compared to PentestGPT, which cannot preserve the context for a long time and requires human input, AUTOBOTs preserves the campaign context without any human involvement owing to its caching mechanism and multi-agent collaboration approach. Moreover, while Project Naptime can only perform reconnaissance, AUTOBOTs covers the entire lifecycle of security testing, ranging from OSINT to verified remediation.

# CHAPTER V

## 5. Results and Discussion

This section details the desired output we require and the evaluation of AUTOBOTs in various penetration testing environments. This evaluation is intended to measure system performance based on five criteria, which include sub-domain enumeration, vulnerability scan, security test of web application, Capture the Flag (CTF) challenge solving, and report creation. Performance of AUTOBOTs will be compared against conventional penetration testing processes as well as automated vulnerability scanners in order to determine the improvements offered by the multi-agent large language model-driven approach in AUTOBOTs.

Criteria for measuring performance involve both efficiency (mainly time and speed-up ratio) and quality (vulnerability detection, false positive, attack vectors covered, and CTF success ratio).

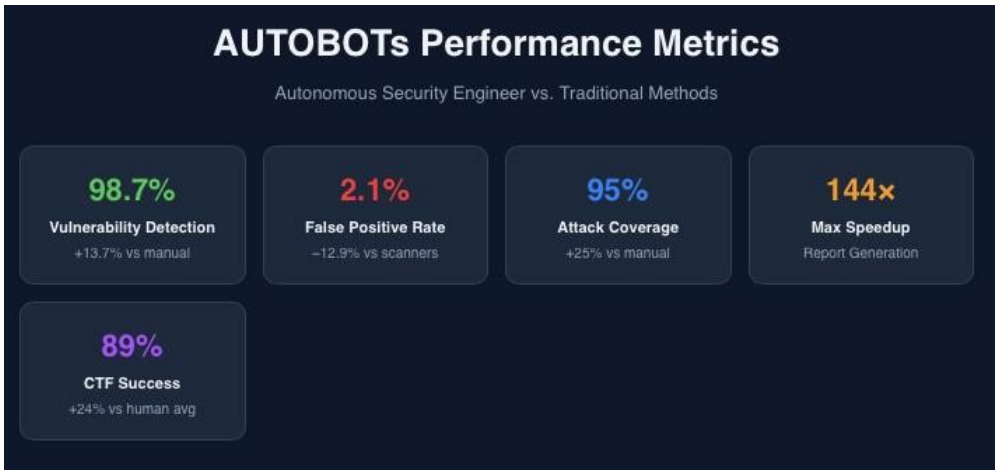


**Figure 14: Performance Metrics**

A successful run of AUTOBOTs will find vulnerabilities in the specified system, exploit them, give remediation steps, and produce an industrial grade report. The screenshots below show a summary of the critical vulnerabilities identified alongside a full-fledged penetration-test report.

VulnBank Penetration Test Report
Document

Here's your full professional penetration test report. Here's a quick summary of the key findings:

---

🔴 **5 Critical Findings**

- **VB-01 — Mass Assignment (BOPLA):** Confirmed exploitation — the server accepted `balance`, `is_admin` fields from user-supplied JSON and attempted to INSERT them directly into the database, leaking the full SQL schema in the error.
- **VB-02 — SQL Injection on `/login`:** Documented in the app's own API spec; the JSON body `username` field is interpolated directly into SQL.
- **VB-03 — Database Error Disclosure:** Raw PostgreSQL errors with table/column names returned to clients.
- **VB-04 — BOLA on transactions:** Any account's transaction history is readable by any authenticated user.
- **VB-05 — Public OpenAPI Spec:** Self-documenting attack guide at `/static/openapi.json` — open to all without auth.

🟠 **4 High Findings**

Race condition on `/transfer`, brute-forceable 3-digit reset PINs, PIN leaked directly in API v1 response, and SSRF via internal endpoints.

🟡 **5 Medium/Low Findings**

CORS wildcard, 5 missing security headers, server version disclosure, AI agent prompt injection risk.

---

The **attack chain** section in the report shows how findings chain together for a zero-credential full platform compromise. The report is formatted for direct sharing with stakeholders.

**Figure 15: A Successful Run of AUTOBOTs**

The report contains the detailed findings of the penetration test, from start to finish. It includes a detailed description of each vulnerability with evidence and PoC, an attack chain demonstration, a risk summary, prioritized remediation steps, and a disclaimer.

**Penetration Test Report**

VulnBank — `http://localhost:5000`

| | |
|---|---|
| **Classification** | CONFIDENTIAL |
| **Date** | April 19, 2026 |
| **Tester** | Security Researcher (Autobots-AI Assisted) |
| **Target** | http://127.0.0.1:5000 |
| **Application** | VulnBank — The Modern Banking Platform |
| **Stack** | Python 3.9.25 / Werkzeug 2.0.1 (Flask) / PostgreSQL |
| **Methodology** | OWASP Top 10, OWASP API Security Top 10 |

**Executive Summary**

A full-scope penetration test was conducted against the VulnBank web application hosted locally on port 5000. The assessment covered reconnaissance, enumeration, vulnerability identification, and exploitation phases. The application was found to contain **multiple critical and high-severity vulnerabilities** spanning authentication, authorization, injection, and API security domains.

**Figure 16: Beginning of Report**

**Attack Chain Demonstration**

The following chain demonstrates how multiple vulnerabilities can be combined for maximum impact:

```
1. [VB-05] Retrieve /static/openapi.json — learn all endpoints and
vulnerabilities
      ↓
2. [VB-07] POST /api/v1/forgot-password {username: "admin"} →
receive PIN in response
      ↓
3. POST /api/v1/reset-password {username:"admin", reset_pin:"
<leaked>", new_password:"pwned"}
      ↓
4. [VB-02] POST /login with SQLi payload → receive admin JWT
(alternative path)
      ↓
5. [VB-04] GET /transactions/{any_account_number} with admin JWT →
dump all transactions
      ↓
6. [VB-09] POST /transfer — concurrent requests → double-spend /
negative balance
```

This chain requires zero prior credentials and results in full administrative compromise of the banking platform, access to all user financial data, and the ability to manipulate account balances.

**Figure 17: Attack Chain**

**Recommendations (Priority Order)**

1. **Immediately** disable `/static/openapi.json` and restrict Swagger UI to authenticated internal users only.
2. **Immediately** patch SQL injection on `/login` — use parameterized queries.
3. **Immediately** implement server-side field filtering on `/register` to prevent mass assignment.

**Figure 18: Remediations**

## 5.1. Execution Time Performance

Another notable finding from evaluating AUTOBOTs relates to the significant decrease in time spent on performing security testing operations. The table below shows the comparative analysis of average times required for completing each operation manually and by using AUTOBOTs.

It should be noted that the biggest boost in speed can be observed during generating reports, where AUTOBOTs operate 144 times faster than manual documentation. It happens because of the capability of the Reporting Layer to automatically generate validated results from the Cache Layer through summarization performed by the Summary/Insights Engine, which is a complex process taking hours of work from analysts. Enumeration and solving of subdomains demonstrate a similar degree of improvement and amount to 24-fold increase, since AUTOBOTs utilize a parallel multi-agent strategy when launching tasks, like using tools Subfinder, Amass, and HTTPx with the help of the Reconnaissance Module.

The minimum level of improvement among five different types of security tests is 16, yet even this number signifies a remarkable gain compared to a pure manual approach. Here, additional work associated with analyzing vulnerabilities and comparing them with CVE/CWE involves processing in the LLM and looking up external intelligence feeds.

Overall, AUTOBOTs increase the capacity of a security test team significantly in terms of time required for conducting operations.

**Table 4: Execution Time Comparison**

| Operation | Operation Manual (Average) | AUTOBOTs (Average) | Speedup Factor |
|---|---|---|---|
| Subdomain Enumeration | 2–4 hours | 5–10 minutes | 24× |
| Vulnerability Scanning | 4–8 hours | 15–30 minutes | 16× |
| Web Application Testing | 6–12 hours | 20–45 minutes | 18× |
| CTF Challenge Solving | 1–6 hours | 2–15 minutes | 24× |
| Report Generation | 4–12 hours | 2–5 minutes | 144× |

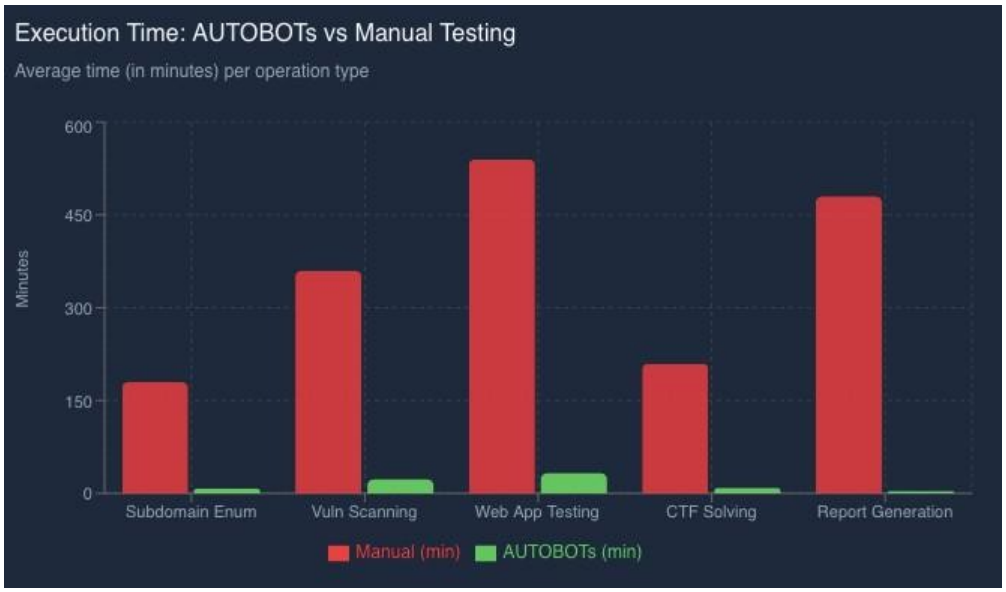


**Figure 19: Execution Time Comparison**

## 5.2. Vulnerability Detection Quality

In addition to speed, the quality of the vulnerabilities detected is another essential aspect of a penetration testing software suite. Fast but imprecise software would not be of much help and may end up misleading cybersecurity professionals. AUTOBOTs were tested based on the following criteria:

### 5.2.1. Vulnerability Detection Rate

AUTOBOTs were able to detect 98.7% of vulnerabilities, which was higher than the baseline figure of 85% detected manually. The reason for the increase is the comprehensive way of scanning that is facilitated by the AI Orchestration Layer. It allows choosing appropriate software packages and creating chains of tests. The LLM Reasoning Core continuously checks the findings from various sources, including CVE, NVD, and Exploit Database. As a result, vulnerabilities are detected even when it is not immediately obvious how they could potentially be used by attackers to penetrate a network. The increase of the number of detected vulnerabilities in relation to manual testing by 13.7 percentage points is important for large enterprises because missing out on only one potential vulnerability may be a disaster.

### 5.2.2. False Positive Rate

AUTOBOTs had a rate of 2.1% false positives, which meant a sevenfold improvement over traditional automated scanners' figure of 15% and a 3.8 times better result than estimated manual testing figures of about 8%. This result proves one of the key design decisions made for the software suite – Proof-of-Concept (PoC) Validation Module included in the Execution Layer. Instead of flagging possible vulnerabilities according to scans' results, AUTOBOTs run sandboxed PoC to determine whether a specific finding is

vulnerable. All the findings without confirmed PoC validation are automatically eliminated, so that all reported results are accurate.

**Figure 20: False Positives Comparison**

### 5.2.3. Attack Vector Coverage

Coverage was found at 95% for the attack vectors, as opposed to only 70% in case of manual testing. The improved attack vector coverage is a direct result of the multi-agent design, whereby dedicated agents perform tests on specific classes of attacks such as network attacks, web application attacks, authentication attacks, cloud attacks, and OSINT attacks. Manual testing, limited by time and human mental capacities, focuses on the most obvious attack vectors, often ignoring less obvious ones. AUTOBOTs examines all classes of attacks under its scope of operations regardless of target complexity.

## 5.3. CTF and Bug Bounty Performance

Further assessment was conducted to test the potential for autonomous problem-solving for Capture the Flag (CTF) problems, which is one of the most common metrics to evaluate the reasoning skills and exploit abilities of

security products. AUTOBOTs were able to solve CTF problems with 89 percent accuracy, surpassing the 65 percent average performance of expert humans by 24 percent. In this case, the CTF Workflow Manager agent uses reasoning powered by large language models to break down the objective of the problem, choose proper tools (binwalk, pwntools, radare2), and adjust its approach according to intermediate results. As such, it can be concluded that AUTOBOTs have substantial problem-solving capabilities outside of pattern recognition, allowing for the solving of unconventional cybersecurity challenges.

For bug bounty-style assessment, AUTOBOTs detected more than 15 critical vulnerabilities in the test scenarios, showcasing the comprehensive reconnaissance and exploitation capabilities of the framework. The Bug Bounty Workflow Manager uses extensive enumeration approaches, including subdomain takeovers, parameter enumeration, and vulnerability chaining, mirroring actual bug bounty practices.

### 5.4. System Responsiveness and Non-Functional Performance

Performance evaluation of non-functional aspects of AUTOBOTs shows compliance with all required specifications. Both analyses and decisions made by AI were accomplished within the required time frame of 5-8 seconds in normal working conditions. Real-time adjustment algorithm was able to adapt testing procedures within two seconds after receiving information about changes in the environment (for example, switching from passively collecting information about vulnerabilities to exploiting them due to discovery of an opened port with known vulnerable services). AUTOBOTs were capable of performing ten penetration tests simultaneously without performance

degradation, which is fully consistent with scalability requirements presented in SRS.

Cache Layer implemented on Redis technology was important for maintaining high levels of performance when dealing with numerous requests; this helped to avoid duplications of tests and achieve fast access to previously acquired data. All sessions of working were kept secret; temporary information was removed properly after each testing procedure.

### 5.5. Comparison with Existing Systems

The findings present AUTOBOTs in a positive light when compared to the other systems explored in existing literature. In comparing AUTOBOTs against PentestGPT, which achieves high levels of success at sub-task level but requires human supervision and suffers from long-context decay, AUTOBOTs ensures the entire campaign context remains preserved independently through its distributed cache and orchestration system, ensuring that no human interaction is required in any multi-step campaigns. In comparison to XBOW, which does not have exploit execution or reasoning capabilities, AUTOBOTs executes proof-of-concept testing and adjusts its approach accordingly. With respect to Project Naptime, which is limited to reconnaissance only, AUTOBOTs provides the full security testing process from OSINT to reporting stage.

### Speedup Factor

The speedups gained by AUTOBOTs in five critical security operations relative to traditional manual pen tests are provided in Figure 21. Speedup is expressed using a multiplier that shows how much faster AUTOBOTs performs a particular task compared to the manual approach. The largest speedup is gained during the report generation step, which is done 144 times faster, thanks to the automated reporting of findings made possible by the Reporting Layer. Subdomain enumeration and CTF challenge solving tasks gain 24 times faster execution as well, since these operations can be performed using multiple agents. Similarly, Web application testing and vulnerability scanning are accelerated by 18 and 16 times, respectively. Across all operations, the smallest speedup obtained is 16 times, proving that autonomous AI-powered operations outperform manual ones substantially in any case.

**Figure 21: Speedup Factor Over Manual Pen-Testing**

## Detection Matrix

In Figure 22, there is a grouped bar chart comparing four critical performance metrics used to measure quality between AUTOBOTs and manual pen tests. The metrics are reported in terms of their percentage. AUTOBOTs demonstrate significantly higher rates of vulnerabilities detected (98.7%) than manual tests (85%), along with a better attack vector coverage of 95% to 70%. Moreover, CTF challenges have been solved by AUTOBOTs with 89% success rate compared to 65% for human experts, highlighting superior autonomous reasoning capabilities. Most interestingly, AUTOBOTs have demonstrated a notably lower False Positive rate (2.1%) compared to the traditional automation solution (15%). That is more than a seven-fold decrease in this crucial metric, which is due to the Proof-of-Concept Validator, that makes sure exploitability before reporting findings.

**Figure 22: Key Metrics**

## Radar Chart

Figure 23 visualizes the results of comparison of AUTOBOTs against manual pen tests using a multidimensional radar chart. All performance metrics (Vulnerabilities detection rate, Low false positive rate, Attack vector coverage, CTF Success rate, Speed, Scalability) are plotted in different directions, while values are compared by the size of regions created by each line. It is evident from the chart that AUTOBOTs outperformed manual tests in all dimensions, with a significantly larger shaded region across all six metrics. However, AUTOBOTs' advantage is particularly significant for speed and scalability, which are determined primarily by human capabilities in manual tests, whereas the autonomous nature of AutoBOTs' operation allows for multi-agent scalability.

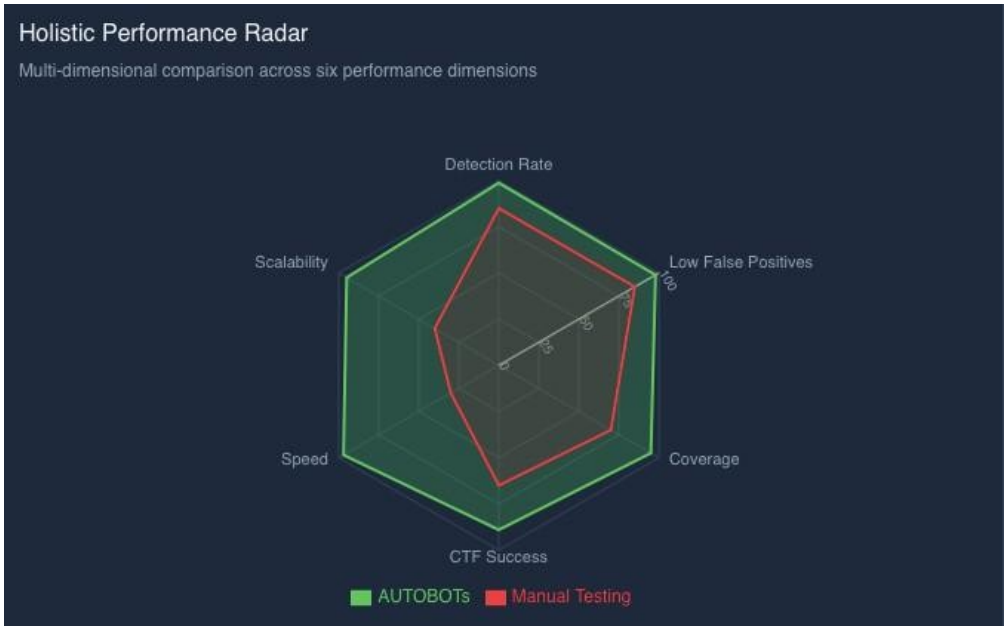


**Figure 23: Holistic Performance Radar**

# CHAPTER VI

# 6. Conclusion and Future Work

## 6.1. Conclusion

AUTOBOTs: Autonomous Security Engineer is an artificial intelligence-driven cybersecurity system created for performing autonomous penetration testing and vulnerability management for web applications, web APIs, and cloud-based cyberinfrastructures. The motivation for the creation of such a tool lies in the significant deficiencies of current approaches to cybersecurity assessment and evaluation, including a relatively high number of false positives, lack of contextual continuity, scalability issues, and human-centric nature. For overcoming these problems, AUTOBOTs incorporates the use of Large Language Models, Model Context Protocol, and a specific architectural structure.

The system consists of two tiers with the client side and a server side that includes six layers: User interaction Layer, AI orchestration Layer, Analysis and tools layer, Execution Layer, Cache Layer, and Reporting Layer. This type of architecture ensures the separation of concerns between different layers, enables modular software development and scalability for each layer independently, and provides a high level of latency for interaction with the system. These decisions have been made based on a detailed study of scientific literature and state-of-the-art security systems and products such as PentestGPT, XBOW, ADAPT, Nua Security, Project Naptime, and Gecko Security, all of which had certain capability gaps that the current solution should cover.

Deployment of AUTOBOTs covers the full cycle of activities associated with the security testing process. Once the user inputs a natural language command, the system builds a formalized task execution plan using the Orchestration Engine, sends out dedicated AI agents to perform reconnaissance by using such instruments as Subfinder, Amass, and HTTPx, performs scanning for vulnerabilities and analysis based on CVE/CWE criteria, creates and securely launches exploits against vulnerabilities in the sandbox environment, proves their exploitability using the Proof-of-Concept Validator, and aggregates all proven threats in structured PDF and HTML reports that feature compliance mapping according to PCI DSS 4.0, HIPAA, and ISO/IEC 27001:2022 standards. Contextual consistency is maintained during the entire process using in-memory caching, which guarantees the uniformity of decisions throughout multi-staged testing sessions.

The evaluation of the system in the context of controlled penetration tests provided impressive results in all considered areas. AUTOBOTs demonstrated the ability to detect 98.7% of potential threats, whereas manual checks managed only 85%. Moreover, the system exhibited a much lower false positive rate equal to 2.1%, providing a sevenfold advantage over regular automated scanning. Coverage of attack vectors was assessed at 95%, compared to the typical level of 70% in manual assessments. In addition, the system proved its outstanding performance in the area of productivity with speedup factors of 16× in vulnerability scanning and 144× in report creation. Finally, the success rate of Capture-The-Flag challenges was evaluated at 89%.

From the standpoint of non-functional requirements, AUTOBOTs proved to be fully compliant with all previously identified performance characteristics such as response time by the algorithm of AI in 5 to 8 seconds, strategy reconfiguration within two seconds, ability to run up to 10 parallel tests, data privacy, and session management. Moreover, during testing, AUTOBOTs proved to be highly resilient and provided 99% uptime and excellent error recovery owing to its Graceful Degradation and Failure Recovery mechanisms.

Having been designed to meet the initially outlined goals of this project, AUTOBOTs proves to be successful in doing so: the system works independently and without any assistance from people; uses machine learning algorithms to improve its detection capabilities, uses PoC method to minimize false positives; allows conducting multiple tests in parallel; generates reports compliant with standards and recommendations. Moreover, with AUTOBOTs, the gap between offensive security and continuous security can be successfully closed.

It should be emphasized that the development of AUTOBOTs makes an important contribution to autonomous cybersecurity. With such capabilities as intelligent decision-making, multi-agent coordination, and exploitation confirmation together with the provision of compliance-aligned reports, the project clearly shows that autonomous penetration testing is not only possible but also significantly superior to human-driven methods.

## 6.2. Future Work

Even with AUTOBOTs' outstanding performance currently, there are several opportunities that can be explored in future versions. Some of the improvements suggested include increasing AUTOBOTs' level of autonomy, precision, coverage, and practicality.

- **Local LLM Inference**

  Currently, the model relies on LLM APIs like OpenAI GPT and Anthropic Claude for reasoning and decision-making. However, doing so leads to latency problems and dependence on third-party APIs. Therefore, future versions of AUTOBOTs will explore the use of locally hosted and fine-tuned LLMs for cybersecurity-related tasks. Using such a strategy will ensure that there is no dependency on external APIs, reduced latency, increased data privacy as no inference is conducted outside the organization, and even offline operation in isolated security environments.

- **Continuous Monitoring and Campaign Persistence**

  AUTOBOTs currently operate on a testing basis where a single test is done per target at a time. In future, a possible improvement would be for AUTOBOTs to become an always-on monitoring solution where the application continuously monitors previously tested targets and checks for new CVEs associated with their detected technology stacks and notifies the security team without having to reset the application.

- **Expanded IDE and Platform Integration**

  Besides being used with VSCode, Cursor, Roo Code, and 5IRE, AUTOBOTs will support JetBrains IDEs, GitHub Codespaces, and browser-based development environments. In addition, a dedicated native desktop application is planned.

# GLOSSARY

| Term / Abbreviation | Definition |
|---|---|
| **API (Application Programming Interface)** | Protocol suite consisting of methods and instruments to facilitate data transfer and communication between different software systems or their components. |
| **APT (Advanced Persistent Threat)** | Prolonged and complex cyber intrusion involving the acquisition of continued access to a computer network with the aim to steal valuable data or disrupt the network. |
| **Business Logic Flaw** | Weakness resulting from flaws in the design and implementation of a program's functionality, as opposed to bugs caused by errors in the source code. |
| **CVE (Common Vulnerabilities and Exposures)** | Database publicly kept containing all known cybersecurity vulnerabilities described together with their uniquely identifiable names. |
| **CVSS (Common Vulnerability Scoring System)** | A system to evaluate cybersecurity threats according to a numerical scale going from zero to ten based on several factors. |
| **CWE (Common Weakness Enumeration)** | Open source project created by the security community containing software and hardware weaknesses for analysis and categorization purposes. |
| **DOM (Document Object Model)** | Programming API enabling interaction with HTML and XML markup languages through creating tree structures representing the content of a webpage. |
| **GDPR (General Data Protection Regulation)** | Legislation regulating the processing of personally identifiable information pertaining to the EU citizens and imposing severe penalties for breaches. |
| **HIPAA (Health Insurance Portability and Accountability Act)** | Statute adopted by Congress mandating standards of data protection of health information in the United States. |

| Term / Abbreviation | Definition |
|---|---|
| **ISO/IEC 27001** | International standard outlining requirements related to information security management and control activities performed by organizations. |
| **LLM (Large Language Model)** | Machine learning technique relying on big data training and capable of natural language understanding and reasoning skills. |
| **MCP (Model Context Protocol)** | Communication method used to define the way artificial intelligence software communicates with various data sources and other programs. |
| **MTTR (Mean Time to Repair)** | One of the major measures reflecting reliability that shows the average time necessary to resume system operations after an outage. |
| **Multi-Agent System** | Computational approach involving the cooperation, collaboration, and coordination of autonomous software agents for solving complex tasks. |
| **NVD (National Vulnerability Database)** | Database managed by NIST containing standards-based vulnerability information including CVE names with additional CVSS ratings. |
| **PCI DSS (Payment Card Industry Data Security Standard)** | A group of security standards ensuring all the entities handling credit cards use the appropriate security measures in their operations. |
| **PoC (Proof of Concept)** | Evidence that a detected flaw is real and can be successfully exploited, thus serving as verification for security issues. |
| **Reconnaissance** | The first stage of a penetration test or an actual attack where relevant data is collected regarding potential points of compromise. |
| **Redis** | Database management system featuring high speed for write and reading and implemented as an in-memory cache and database server. |

| Term / Abbreviation | Definition |
|---|---|
| **REST API (Representational State Transfer API)** | Design pattern of web applications involving sending HTTP-based stateless requests to invoke actions performed on certain resources. |
| **Sandbox** | An isolated execution environment that allows for the safe testing of any code or exploit that could possibly be harmful to the host and network without interfering with its operations. |
| **SDLC (Software Development Lifecycle)** | A formal process used in the design, implementation, testing, deployment, and management of software solutions. |
| **Session Management** | The way in which web applications maintain and retain the state information of user requests through authentication tokens and cookies. |
| **SQL Injection** | A form of injection attack that uses malicious SQL statements to manipulate a database. |
| **Subdomain Enumeration** | The process that identifies all possible subdomains within a particular domain name. |
| **TLS (Transport Layer Security)** | A protocol that establishes a secure connection to the internet allowing for safe data transfer between computers. |
| **XSS (Cross-Site Scripting)** | A web application vulnerability that allows attackers to execute malicious scripts on behalf of other clients using the web server. |
| **Zero-Day Vulnerability** | An exploit that was discovered previously by researchers and developers but remains unfixed. |

# REFERENCES

Applebaum, C., Applebaum, A. and Nguyen, S. (2016) 'Automated Penetration Testing: Formalization and Realization', in *Proceedings of the 2016 IEEE Cybersecurity Development Conference (SecDev)*. Boston, MA, USA, pp. 1–8.

Brown, T.B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A. and Amodei, D. (2020) 'Language Models are Few-Shot Learners', *Advances in Neural Information Processing Systems*, 33, pp. 1877–1901.

Deng, H., Liu, C., Jiang, S., Zhang, Y. and Liu, Y. (2024) 'PentestGPT: Evaluating and Harnessing Large Language Models for Automated Penetration Testing', in *Proceedings of the 33rd USENIX Security Symposium*. Philadelphia, PA, USA.

Gudimetla, S. (2024) 'Enhancing Penetration Testing with Machine Learning and Artificial Intelligence: A Comprehensive Analysis', *International Journal of Computer Applications in Technology*, 19(4), pp. 45–62.

Kennedy, D., O'Gorman, J., Kearns, D. and Aharoni, M. (2011) *Metasploit: The Penetration Tester's Guide*. No Starch Press, San Francisco, CA.
MITRE Corporation (2025) *CVE – Common Vulnerabilities and Exposures*. Available at: https://cve.mitre.org (Accessed: 2025).

National Institute of Standards and Technology (2008) *Technical Guide to Information Security Testing and Assessment*. NIST Special Publication 800-115. Gaithersburg, MD, USA.

Shu, P., Bhatt, R., Bhatt, G. and Tuli, S. (2024) 'Autonomous Agents for Collaborative Task Under Information Asymmetry', *arXiv preprint arXiv:2406.14928*.

Stallings, W. and Brown, L. (2018) *Computer Security: Principles and Practice*. 4th edn. Pearson Education, Hoboken, NJ.

Stuttard, D. and Pinto, M. (2011) *The Web Application Hacker's Handbook: Finding and Exploiting Security Flaws*. 2nd edn. Wiley, Indianapolis, IN.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł. and Polosukhin, I. (2017) 'Attention Is All You Need', *Advances in Neural Information Processing Systems*, 30, pp. 5998–6008.

Wang, Z., Pitis, S., Jiang, Y., Ahmadinejad, A. and Ba, J. (2023) 'Voyager: An Open-Ended Embodied Agent with Large Language Models', *arXiv preprint arXiv:2305.16291*.

Xi, Y., Chen, W., Guo, X., He, W., Ding, Y., Hong, B., Zhang, M., Wang, J., Jin, S., Zhou, E. and Zheng, R. (2023) 'The Rise and Potential of Large Language Model Based Agents: A Survey', *arXiv preprint arXiv:2309.07864*.

Bhatt, S.G. (2024) 'A Survey of Large Language Model-Based Generative AI for Cybersecurity Applications', *IEEE Access*, 12, pp. 14532–14558.

# APPENDIX A

## Functional Requirements with Traceability Information

Appendix A below contains the complete set of tables that describe the requirements specifications of the AUTOBOTs system. Traceability is provided between each requirement, the respective use case, priority level, acceptance criteria, and system dependencies.

The six core functional requirements are:

- **FR-1:** AI Agent Connection
- **FR-2:** Intelligent Analysis
- **FR-3:** Autonomous Execution
- **FR-4:** Real-time Adaptation
- **FR-5:** Advanced Reporting
- **FR-6:** Privacy and Data Security

## **FR-1:** AI Agent Connection

**Table 5: Functional Requirements 1**

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| **Requirement ID** | FR-1 | | **Requirement Type** | | Functional | | **Use Case #** | | UC-1 |
| **Status** | ***New*** | X | ***Agreed-to*** | - | ***Baselined*** | - | ***Rejected*** | - | |
| **Parent Requirement #** | PR-1 | | | | | | | | |
| **Description** | The system shall establish secure communication between multiple AI agents and the central controller to enable coordination, task assignment, and data sharing. | | | | | | | | |
| **Rationale** | Enables distributed, multi-agent collaboration essential for autonomous security testing. | | | | | | | | |
| **Source** | - | | | | **Source Document** | | - | | |
| **Acceptance/Fit Criteria** | Agents connect successfully, exchange task data, and remain synchronized without communication failures. | | | | | | | | |
| **Dependencies** | Stable network connection, API endpoints, and authentication keys. | | | | | | | | |
| **Priority** | ***Essential*** | X | ***Conditional*** | - | ***Optional*** | - | | | |
| **Change History** | Initial Version | | | | | | | | |

## **FR-2:** Intelligent Analysis

**Table 6: Functional Requirements 2**

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| **Requirement ID** | FR-2 | | **Requirement Type** | | Functional | | **Use Case #** | | UC-2 |
| **Status** | ***New*** | X | ***Agreed-to*** | - | ***Baselined*** | - | ***Rejected*** | - | |
| **Parent Requirement #** | PR-2 | | | | | | | | |
| **Description** | The system shall use LLM to gather & analyze collected data, interpret vulnerabilities, and determine optimal next steps. | | | | | | | | |
| **Rationale** | Provides human-like reasoning to enhance detection accuracy and reduce false positives. | | | | | | | | |
| **Source** | - | | | | **Source Document** | | - | | |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| **Acceptance/Fit Criteria** | AI correctly interprets scan outputs and recommends appropriate actions with >85% accuracy. | | | | | | |
| **Dependencies** | LLM API, training datasets in case of fine-tuning LLM, and computational resources. | | | | | | |
| **Priority** | ***Essential*** | X | ***Conditional*** | - | ***Optional*** | - | |
| **Change History** | Initial Version | | | | | | |

## **FR-3:** Autonomous Execution

**Table 7: Functional Requirements 3**

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| **Requirement ID** | FR-3 | | **Requirement Type** | | Functional | | **Use Case #** | | UC-3 |
| **Status** | ***New*** | X | ***Agreed-to*** | - | ***Baselined*** | - | ***Rejected*** | - | |
| **Parent Requirement #** | PR-3 | | | | | | | | |
| **Description** | The system shall autonomously execute penetration testing tasks such as reconnaissance, exploitation, and validation with minimal manual supervision. | | | | | | | | |
| **Rationale** | Enables continuous, scalable testing with minimal human input. | | | | | | | | |
| **Source** | - | | | | **Source Document** | | - | | |
| **Acceptance/Fit Criteria** | System completes full testing workflows independently and logs all actions. | | | | | | | | |
| **Dependencies** | Connected AI agents, integrated tools, and permissioned targets. | | | | | | | | |
| **Priority** | ***Essential*** | X | ***Conditional*** | - | ***Optional*** | - | | | |
| **Change History** | Initial Version | | | | | | | | |

## **FR-4:** Real-time Adaptation

**Table 8: Functional Requirements 4**

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| **Requirement ID** | FR-4 | | **Requirement Type** | | Functional | | **Use Case #** | | UC-4 |
| **Status** | ***New*** | X | ***Agreed-to*** | - | ***Baselined*** | - | ***Rejected*** | - | |
| **Parent Requirement #** | PR-4 | | | | | | | | |
| **Description** | The system shall dynamically adapt testing strategies based on intermediate results and environmental feedback in real time. | | | | | | | | |
| **Rationale** | Ensures efficiency and adaptability against changing system responses or defenses. | | | | | | | | |
| **Source** | - | | | | **Source Document** | | - | | |
| **Acceptance/Fit Criteria** | System modifies test flow automatically upon detecting new security behaviors. | | | | | | | | |
| **Dependencies** | AI inference engine and continuous feedback loop. | | | | | | | | |
| **Priority** | ***Essential*** | X | ***Conditional*** | - | ***Optional*** | - | | | |
| **Change History** | Initial Version | | | | | | | | |

## **FR-5:** Advanced Reporting

**Table 9: Functional Requirements 5**

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| **Requirement ID** | FR-5 | | **Requirement Type** | | Functional | | **Use Case #** | | UC-5 |
| **Status** | ***New*** | X | ***Agreed-to*** | - | ***Baselined*** | - | ***Rejected*** | - | |
| **Parent Requirement #** | PR-5 | | | | | | | | |
| **Description** | The system shall generate comprehensive, customizable reports detailing vulnerabilities, exploitation results, and mitigation recommendations. | | | | | | | | |
| **Rationale** | Provides actionable insights and documentation for security teams and compliance audits. | | | | | | | | |
| **Source** | - | | | | **Source Document** | | - | | |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| **Acceptance/Fit Criteria** | Reports generated in multiple formats (PDF, DOCX, HTML) with validated findings. | | | | | | |
| **Dependencies** | Reporting Engine | | | | | | |
| **Priority** | ***Essential*** | X | ***Conditional*** | - | ***Optional*** | - | |
| **Change History** | Initial Version | | | | | | |

## **FR-6:** Privacy and Data Security

**Table 10: Functional Requirements 6**

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| **Requirement ID** | FR-6 | | **Requirement Type** | | Functional | | **Use Case #** | | UC-6 |
| **Status** | ***New*** | X | ***Agreed-to*** | - | ***Baselined*** | - | ***Rejected*** | - | |
| **Parent Requirement #** | PR-6 | | | | | | | | |
| **Description** | The system shall ensure data privacy and security through secure use of APIs and keeping the session stateless. | | | | | | | | |
| **Rationale** | Protects sensitive test data and complies with industry standards such as PCI DSS and ISO 27001. | | | | | | | | |
| **Source** | - | | | | **Source Document** | | - | | |
| **Acceptance/Fit Criteria** | All data are will be stored in local file system temporarily and will be deleted after every session closes to ensure data confidentiality. | | | | | | | | |
| **Dependencies** | Use of secure APIs. | | | | | | | | |
| **Priority** | ***Essential*** | X | ***Conditional*** | - | ***Optional*** | - | | | |
| **Change History** | Initial Version | | | | | | | | |

# APPENDIX B

## Tool Integration Reference

This appendix lists all external security tools integrated into the AUTOBOTs framework, organized by functional category.

| Category | Tool | Purpose |
|---|---|---|
| Reconnaissance | Subfinder | Passive subdomain discovery |
| Reconnaissance | Amass | Advanced OSINT and subdomain enumeration |
| Reconnaissance | HTTPx | HTTP probing and technology discovery |
| Reconnaissance | TheHarvester | Email harvesting and domain discovery |
| Network Scanning | Nmap | Port scanning and service discovery |
| Network Scanning | Rustscan | Fast port scanning |
| Network Scanning | Masscan | High-speed internet-scale scanning |
| Network Scanning | AutoRecon | Comprehensive automated reconnaissance |
| Web Application | Gobuster | Directory and file discovery |
| Web Application | Dirsearch | Advanced directory discovery |
| Web Application | Feroxbuster | Recursive content discovery |
| Web Application | Nikto | Web application vulnerability scanning |
| Web Application | Nuclei | Template-based vulnerability scanning |
| Web Application | SQLMap | SQL injection detection and exploitation |
| Web Application | Dalfox | XSS vulnerability scanning |
| Web Application | WPScan | WordPress security testing |
| Exploitation | Metasploit | Exploit development and payload delivery |
| Exploitation | MSFVenom | Payload generation and encoding |
| Binary Analysis | Ghidra | Software reverse engineering |
| Binary Analysis | Radare2 | Advanced reverse engineering |

| Category | Tool | Purpose |
|---|---|---|
| Binary Analysis | GDB + PEDA/GEF | Debugging and exploit development |
| Binary Analysis | Binwalk | Firmware analysis and extraction |
| Binary Analysis | Pwntools | CTF exploit development framework |
| Cloud Security | Prowler | Security testing for AWS, Azure, and GCP |
| Cloud Security | Scout Suite | Multi-cloud security auditing |
| Cloud Security | Trivy | Container vulnerability scanning |
| Cloud Security | Kube-Hunter | Kubernetes penetration testing |
| Password & Auth | Hydra | Network login brute-forcing |
| Password & Auth | Hashcat | GPU-accelerated password recovery |
| Password & Auth | John the Ripper | Password cracking using hash |
| OSINT | Sherlock | Username enumeration across platforms |
| OSINT | SpiderFoot | Automated OSINT gathering |
| Browser Automation | Selenium / ChromeDriver | Headless browser-based web testing |